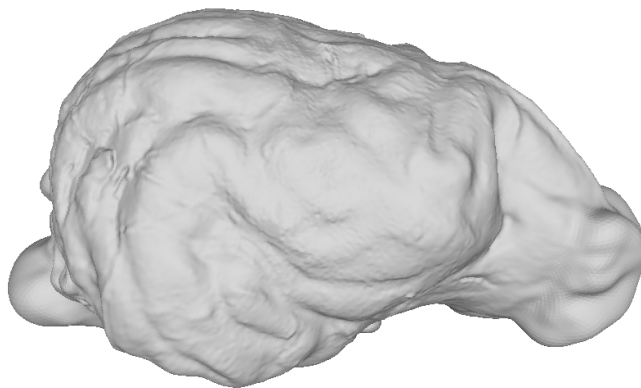


TECHNICAL UNIVERSITY OF DENMARK



A Mesh-Based Approach to Digital Endocasting from Micro-CT-Scans of Animal Skulls

BACHELOR THESIS



s224389 - Christian Lundgaard Bjerregaard
s224362 - Aske Thomsen Rove

Supervisors:
Hans Martin Kjer,
Christian Møller Mikkelsen,
Christy Anna Hipsley &
Patrick Møller Jensen

June 11, 2025

Abstract

This thesis investigates methods for creating detailed digital casts of wolf endocrania, called endocasts, from μ CT-scans. We evaluate two existing methods, Endomaker and SlicerMorph, identifying shortcomings such as fragility and limited guidance for less technical users. Therefore, we propose a novel mesh-based method for digital endocast creation inspired by the Round Cut algorithm. Our approach involves estimating the center point of the endocranium, from which a spherical mesh iteratively expands out using graph-cut segmentation. Our evaluation demonstrates that our proposed method effectively produces accurate and visually detailed endocasts, comparable in runtime to the existing methods, but with a superior result in the visual quality of the endocasts. However, scalability remains limited for volumes significantly exceeding 1000^3 voxels, and intricate internal structures, such as narrow crevices, remain a challenge to capture in high detail. Despite these limitations, our approach serves as a very promising proof of concept. We highlight ways in which our method could be improved upon in future work to optimize it and generalize it to other species, thus potentially benefiting broader applications in paleoneurology and evolutionary studies.

Contents

1	Introduction	3
2	Data	6
2.1	μ CT-scans	6
2.2	Supplied data	6
2.3	The NIfTI file format	7
3	Existing Methods	8
3.1	Endomaker	8
3.1.1	Introduction to Endomaker	8
3.1.2	Testing Endomaker	9
3.1.3	Results from Endomaker	9
3.2	SlicerMorph	10
3.2.1	Introduction to SlicerMorph	10
3.2.2	Testing SlicerMorph	10
3.2.3	Results from SlicerMorph	11
3.3	Discussion of and conclusion on existing methods	11
4	Relevant Theory	12
4.1	Downsampling	12
4.2	Detection of a smooth layer in an image	12
4.2.1	Construction of the graph	13
4.2.2	Properties of a minimum s - t cut on the graph	17
5	Our novel method	19
5.1	Segmenting bone structure of interest in volume	19
5.1.1	Locating bone	19
5.1.2	Finding the largest contiguous bone-structure	20
5.2	Creating an axis-aligned volume	21
5.2.1	Rotation of the volume	21
5.2.2	Resampling of the volume	22
5.3	Estimating the center point of the endocranium	23
5.3.1	Using the convex hull to limit the search space	23
5.3.2	Finding candidate center points and picking the right one	23
5.4	Generating the endocast	25
5.4.1	Preparing a spherical mesh at the center of the endocranium	26
5.4.2	Sampling along mesh normal vectors	27
5.4.3	Moving mesh vertices closer to the inside surface of the neurocranium	29
5.4.4	Difference between the first iteration and the following ones	31
6	Results	34
6.1	Results analyzed	34
6.2	Runtimes and mesh resolution	37
7	Discussion	39
7.1	Did we solve the problem?	39
7.1.1	Results	39
7.1.2	Strengths of our method	39
7.1.3	Shortcomings of our method	39
7.1.4	Comparison to Endomaker and SlicerMorph	40
7.2	Alternative methods and future work	41
7.2.1	Method refinements	41

CONTENTS

7.2.2	Alternative approaches	43
7.2.3	Running locally	44
7.3	Perspective	44
8	Conclusion	45
9	Appendix	48
A	SlicerMorph Pictures	48
A.1	SlicerMorph result 1	48
A.2	SlicerMorph result 2	49
A.3	SlicerMorph result 3	50
B	Results	51
B.1	Settings used in time comparison table	51

1 Introduction

Endocasting is the process of creating detailed models of the endocranium, which is the internal cavity of an animal's skull in which the brain resides. These models, called endocasts, allow for scientists to study the anatomy and evolution of different species' brains, giving insights into their cognitive abilities, sensory functions, and developmental patterns. It is important to distinguish the endocranium from the neurocranium. The neurocranium is the broader skull structure that protects the brain, and thus an endocast is a cast of the cranial cavity inside of the neurocranium. These structures provide essential clues about the biology of both extinct and living animals. Skulls hold particular significance for evolutionary and paleontological research, as their robust structure enhances preservation potential, allowing them to remain for very long periods of time underground [1].

The pioneering of the field of paleoneurology, which is the study of the evolution of the brain lying at the interface of neurology and paleontology, is largely credited to German Ottilie ("Tilly") Edinger (1897-1967). Before her work, the history of the vertebrate brain was reconstructed almost only by anatomists who would compare the soft tissue brains of living amphibians, reptiles, fish, birds, and mammals. With Edinger's 1929 book *Die fossilen Gehirne* (Fossil Brains) she formally defined a new field of inquiry. Edinger gathered together references to many endocranial casts (endocasts) that had been treated as isolated curiosities in earlier texts, organized them taxonomically, and then summarized the inferences that could be drawn from them [2], [3].

Historically, endocasts have been produced using physical techniques. Researchers would typically fill the endocranial cavity with materials like plaster, latex, or silicone. While groundbreaking at the time, these physical methods had limitations. For instance, they could damage delicate fossils, were labor intensive and time consuming, and often had a suboptimal resolution in the final product. These problems meant that scientists could not take full advantage of the skull fossils found, meaning that endocasts were not as useful as they could have been for comparing species or studying brain evolution. These difficulties have now been largely overcome through digital technologies for creating endocasts [1], [4].

Since the 1990's evolutionary morphology has witnessed an explosion of digital techniques for observing, analyzing and interpreting anatomical information. The impact of these technologies has been transformative in the field of comparative neuroanatomy where the various forms of computed tomography (CT) are widely used across vertebrates to visualize the endocranial space and to construct digital endocasts. These days CT imaging has become standard practice in paleontology, making this data and information more available than ever [5]. This offers scientists a new way of viewing the endocranium with high resolution and without damaging fossils. Therefore, the modern standard for endocasting is through digital methods on various CT-scans.

Different methods exist for digital endocast creation. In this thesis, we will look into two specific ones: Endomaker and SlicerMorph, which take two different approaches to the problem of generating digital endocasts [6], [7]. The current methods are not perfect, and thus, there is more work to do in this field to improve upon the results that can be obtained when doing digital endocasting, as well as the accessibility of the tools for less experienced users. We were aware of methods that seemed promising for working well in solving the task of creating detailed and accurate digital endocasts from CT-scans in a user-friendly way - enabling more researchers to draw use of it - which was of interest to look further into.

The goal and scope of this thesis is the following: We will be evaluating the two aforementioned existing methods for digital endocast creation, assessing their strengths and shortcomings. After this, we will introduce our own novel approach to digital endocast creation, which aims to solve some of the found shortcomings of the existing methods. This thesis will be focusing on generating digital endocasts from μ CT-scans (micro-CT-scans) of the *Canis lupus* species,

commonly known as the gray wolf, or in this thesis simply wolves. μ CT-scans are CT-scans of very high resolution, which is useful for analyzing smaller species and when great detail is wanted. We aim to lay the ground work and show a proof of concept for a new approach to creating digital endocasts inspired by existing computational methods from other fields. We are therefore not expecting a perfect and flawless final version from this thesis. Though we will focus on wolves in this thesis, the aim is to create a method that, with work, will be able to generalize to more species, making it a valuable tool for biologists working in relevant fields.

One of the challenges faced in endocasting of skulls is that the endocranium is not a closed structure. This can be even worse in fossils where bones may not be intact. An illustration of a wolf skull can be seen in Figure 1. The red square indicates the *foramen magnum*, latin for "great hole", which allows for the brainstem to exit through the skull and become the spinal chord. Therefore, any methodology that aims to cast the endocranium, should be able to deal with these topological properties of the skull. This is why an approach based on volumes, meshes, and graphs seems promising, as the latter has good ways of incorporating geometrical constraints and knowledge [8].

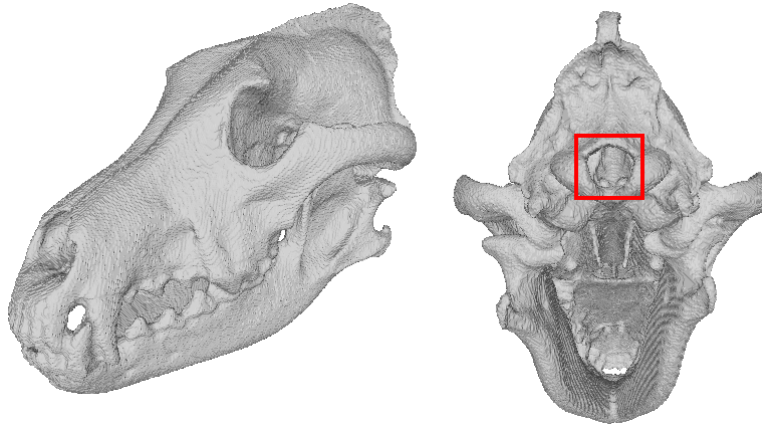


Figure 1: A mesh of a wolf skull. The *foramen magnum* is marked by a red square.

Figure 2 shows a digital wolf endocast with approximate regional brain areas marked. Note the olfactory bulbs, as they will also become relevant later on when developing and testing our proposed method.

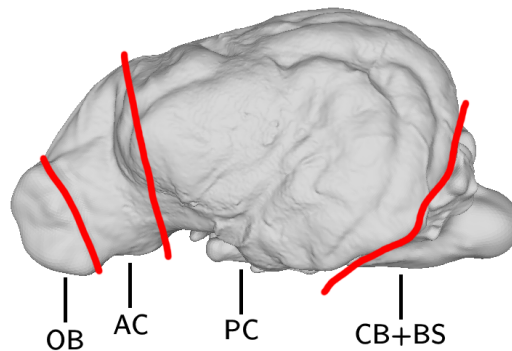


Figure 2: An illustration of a wolf endocast showing approximate regional brain areas: olfactory bulbs (OB), anterior cerebrum (AC), posterior cerebrum (PC), and combined cerebellum and brainstem (CB + BS). Inspired by fig. 3 in [9].

Our novel method seeks to utilize a modified version of the Round Cut algorithm proposed by Dahl et al. [10] to detect the closed surface that is the endocranium. From having a μ CT-scan of a wolf, we first transform the volumetric data to a new coordinate system, in which the standard anatomical axes of the skull align with the axes of the volume. Then, we locate the center point of the endocranium, which will act as a seed point for a mesh that will iteratively expand out and eventually create a cast of the endocranium. An illustration of these overall steps can be seen in Figure 3.

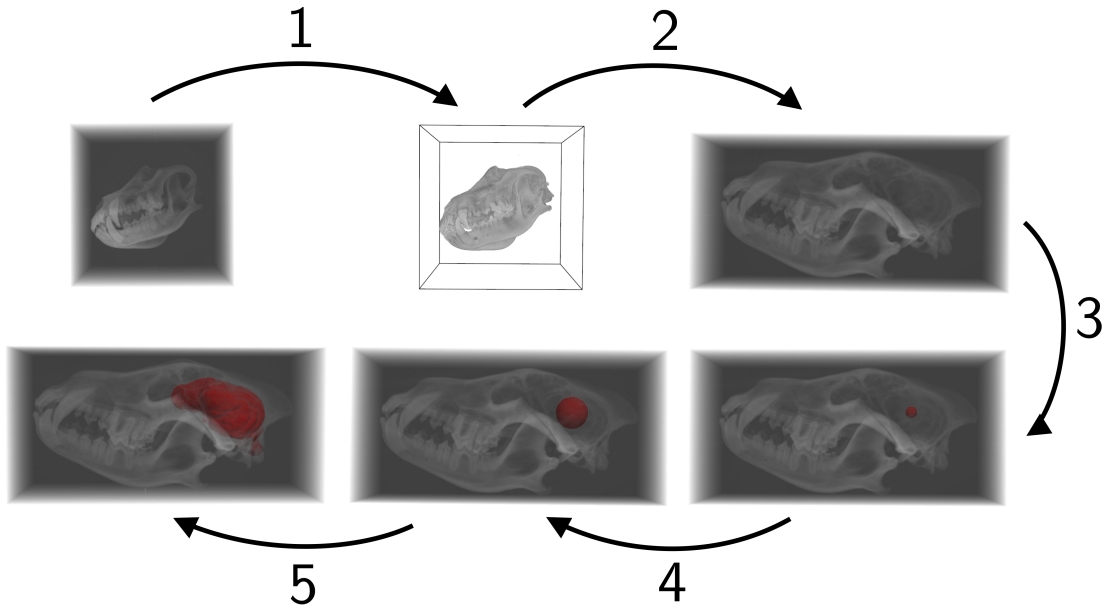


Figure 3: An illustration of the pipeline in our novel method for digital brain endocasting. 1) Locate skull in volume. 2) PCA align to anatomical axes. 3) Find estimated center point. 4) Initialize mesh icosphere. 5) Expand mesh iteratively.

The structure of this thesis is as follows. Section 2 will cover the data used. Our analysis of the two existing methods is covered in section 3. Section 4 describes relevant theory that is necessary to understand the methods and results. The reader can either read this section from start to finish, or read the proper subsections when they become necessary in order to understand our proposed method. Our novel method is described in section 5. Then, different results obtained using our method are shown, and a discussion of these and the methods follows. Finally, we give a conclusion.

2 Data

2.1 μ CT-scans

A μ CT-scan is a 3D imaging technique utilizing X-rays to see inside an object, without physically cutting into them. It is similar to hospital CT or "CAT" scan imaging but on a small scale with greatly increased resolution. Samples can be imaged with pixels sizes as small as 100 nanometers and objects can be scanned as large as 200 millimeters in diameter. Due to the increased resolution μ CT-scans can be used to study the interior structure of both material and biological samples. The quantitative information obtained from μ CT-scanning can only be obtained from 3D images, and 3D digital models created from μ CT-virtual slices allow scientists to measure any parameters for comparison in before-and-after studies.

These unique features of μ CT-scanning allow scientists and researchers to look at the morphology of a sample and study features such as structure/bone thickness, volume fractions, defect analysis, voids (endocrania), etc. [11].

2.2 Supplied data

For this project, we were supplied with a dataset of μ CT-scans of wolf skulls. They were supplied to us by Associate professor Christy Anna Hipsley from the department of Biology at The University of Copenhagen. In our project we primarily focus on 4 specific wolf skulls stemming from the larger collection of μ CT-scans. Each wolf μ CT-scan has a unique name/identifier. The ones used in this thesis are named BM_2600, NRM_20105021, NRM_A586548 and UAM_47189. These will be mentioned when appropriate. Particularly in the results section.

The μ CT-scans were supplied in the form of .vol files (*volume data files*) paired with .vgi files (*volume graphics information files*). A .vol file is a raw 3D volumetric data file, that stores the actual voxel intensities usually from CT or μ CT-scans. The image data can be binary-encoded, compressed or uncompressed and stored as 8-, 16- or 32-bit integers or floats. The .vgi file stores relevant meta-data associated with the scan, think of it like a header or configuration file that tells a viewer how to interpret the .vol file [12]. For our interest and purpose the information we needed from the .vgi files were:

- The voxel spacing ($\approx 0.12 \frac{\text{mm}}{\text{voxel}}$).
- The volume dimensions ($2000 \times 2000 \times 2000$ voxels).
- The affine matrix.

An example of what one of the supplied μ CT-scans could look like in a 3D rendering can be seen in Figure 4 on the next page.

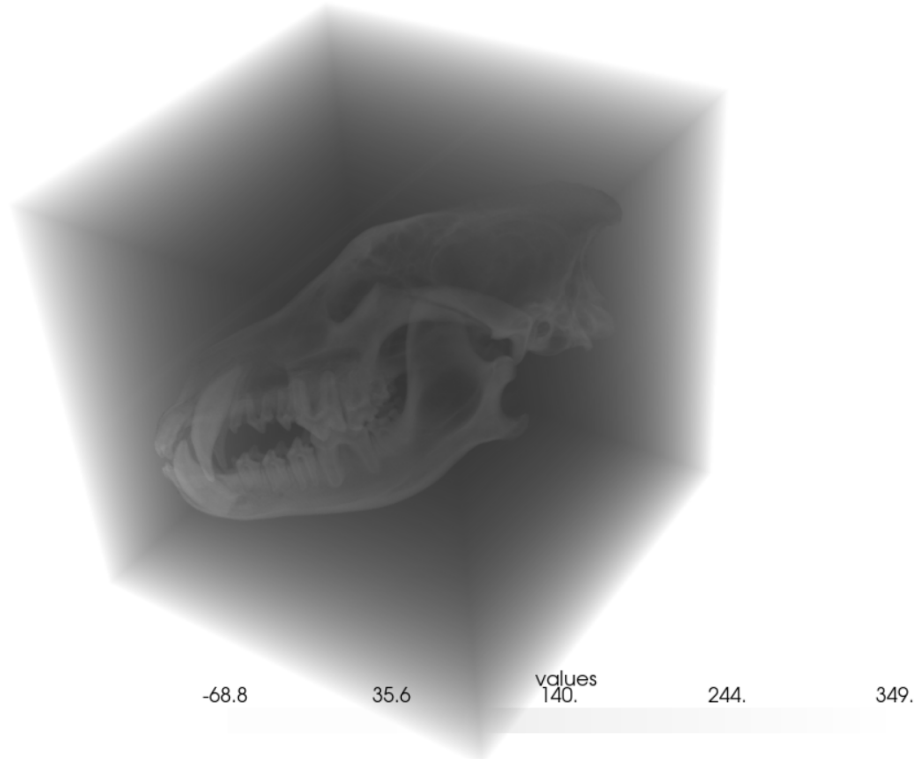


Figure 4: A rendering of a wolf skull μ CT-scan file using PyVista [13]. The skull of a wolf can vaguely be seen.

2.3 The NIfTI file format

In this project, we convert the original `.vol` and `.vgi` files into the NIfTI (Neuroimaging Informatics Technology Initiative) format (`.nii`) as a preprocessing step [14]. This decision is motivated by several factors, one of which being the fact that NIfTI is an open and widely adopted format in the neuroimaging and biomedical imaging communities. Converting our μ CT-scan to NIfTI thus also allows us to use already established tools such as ITK-SNAP for analysis [15]. A NIfTI file stores all the data as a single file, so no metadata file is needed. From the NIfTI file the data used is

- The discrete volume containing the image data.
- The 4×4 affine matrix mapping the array coordinates to coordinates in some reference space (world coordinate space) to give an understanding of orientation and scale.
- The image metadata in the form of a header.

3 Existing Methods

As part of this project, we tested two already existing methods for creating digital endocasts, namely Endomaker and SlicerMorph [6], [7]. We performed some preprocessing to reduce possible noise in the μ CT-scans (BLOB-analysis), to reduce the file size (downsampling), and to align the skull in the μ CT-scan in a more easy to interpret way (PCA-resampling). The exact details of these methods are not of great importance right now. They will be covered in more detail in sections 4.1, 5.1 and 5.2. The main motivation for doing these preprocessing steps is to supply the existing methods with as clean data as possible to improve the chances of good results. The tests in this section will all be on downsampled volumes of dimensions $500 \times 500 \times 500$ voxels.

3.1 Endomaker

3.1.1 Introduction to Endomaker

Endomaker by A. Profico et al. is a mesh-based approach to creating endocasts [6], as it takes in a mesh of the skull as its input. The method is demonstrated to work on a variety of skulls including wolves. Endomaker is available through a library called "Arothron" in R and the Endomaker algorithm both produces an endocast and an endocranial volume estimation. The endocasts is created in the following steps:

1. A mesh of the skull is given as input.
2. A bounding box of this mesh is defined.
3. The bounding box is converted into a 3D binarized grid of voxels where an intensity of 1 indicates if just one vertex of the voxel intersects with the mesh and an intensity of 0 indicates it does not.
4. The local density of the volume is calculated using distance fields. This helps differentiate between high density and low density regions of the skull.
5. An alpha shape (the smallest polygon enveloping the skull) is calculated as to include the entire volume occupied by the mesh. The 0-valued voxels inside the alpha shape will correspond to the cranial internal cavities.
6. Within the internal cavities find the largest volume of contiguous 0-valued voxels. This will be the endocranial cavity.
7. Find the barycenter of this cavity to locate and automatically place a single point of view (POV).

When the POV has been placed a modified version of the AST-3D (Automatic Segmentation Tool for 3D objects) tool is applied. AST-3D performs digital reconstruction of anatomical cavities but requires view points to be preset. It does this by shooting rays from the set points of view, detecting the first triangles of the mesh it hits in each direction. More details can be found at [16]. The modified steps are:

1. A raw version of the endocast is acquired by running AST-3D with the found POV from before.
2. A new alpha shape is calculated to approximate the 3D space occupied by the endocast.
3. The alpha shape is scaled by a given factor (0.5 as default) to avoid unintended addition of extra "material" (bone) to the endocast.
4. A number of user-defined evenly spaced POVs are placed inside the cavity.
5. At last the standard AST-3D algorithm is run to produce the endocast.

Based on the user-defined POVs, the AST-3D algorithm casts rays outwards in all directions from each internal POV. By doing this the algorithm automatically detects the triangles of the input mesh that are visible from the POV's. Any triangle that is first hit along a ray from the POV is marked as part of the cavity's surface. It is important to do it for many POVs as each POV may reveal different portions of the interior surface. The algorithm then combines the results by taking the union of all the triangles seen from any interior viewpoints. This will simulate a scan from the inside of the cavity. The set of found triangles thus make up the estimation of the endocast.

3.1.2 Testing Endomaker

Our input data was first preprocessed, as described briefly at the start of this section. Then, we applied the marching cubes algorithm from **Scikit-Image** on the preprocessed volume to make a mesh from the volume and converted it to a `.ply` file. Figure 5 shows a resulting mesh from this process which can then be used as the input for Endomaker [17].



Figure 5: The resulting mesh after applying the marching cubes algorithm on a $500 \times 500 \times 500$ voxel volume of the NRM_20105021 μ CT-scan. The mesh has roughly 1.3 million vertices and 2.5 million faces.

We tested Endomaker in three stages: the first one being the aforementioned, in the second we used binary morphology operations to try to improve results, and in the third we cropped tightly around the endocranium, to ensure that Endomaker would find the endocranium. All these examples have been tested on a consumer grade laptop with 16 GB of RAM and an AMD Ryzen™ 5 7530U-processor where each sample took a matter of minutes to run.

3.1.3 Results from Endomaker

Endomaker works entirely automatically and can take the standard mesh type files (`.stl`, `.ply` or `.obj`) without any other input, so it is easy to use. From loading the library to having a viewable endocast, with our data only takes between 2-3 minutes.

However, despite being a straightforward implementation, we encountered some challenges. We primarily found it sensitive to input (more/less smoothing could drastically change the outcome) and we got warnings or error codes that we could not figure out how to fix, even when searching the web. Additionally, the error "*Warning: did not converge in 10 iterations*" seemed to send the algorithm into two different directions: sometimes we would get an incorrect endocast (it would extract the inside of the jaw bone) or we would get the actual endocast, see Figure 6.

We discovered that we could significantly increase our chances of us getting the correct output, by cropping the mesh. This meant discarding everything that we knew it shouldn't look for, so that Endomaker could only locate one internal cavity - namely the endocranial cavity. This came with the added bonus of getting a quicker result. In the end we didn't find the meshes very

satisfactory as the meshes would have multiple holes and be incomplete. As seen in Figure 6b), Endomaker was unable to include the olfactory bulb as part of the endocast [9].

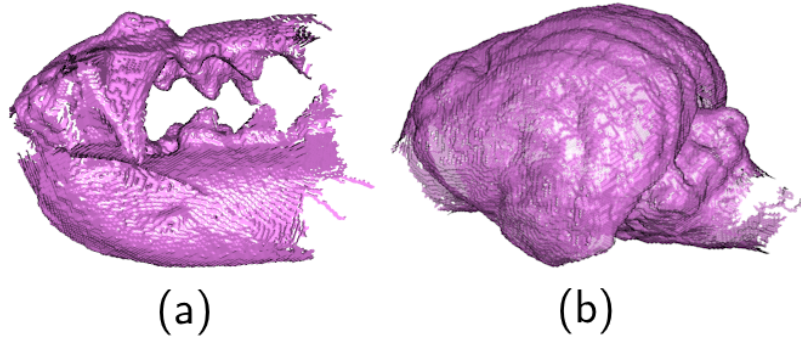


Figure 6: (a) Unwanted jaw extraction. (b) The endocast does not capture the olfactory bulbs (on the left-hand side), and it is not at all watertight, as is clearly seen near the *foramen magnum* (on the right-hand side).

3.2 SlicerMorph

3.2.1 Introduction to SlicerMorph

SlicerMorph is an add-on to the program 3D-Slicer [18]. SlicerMorph is used to segment endocrania and it works entirely without any user interaction from when the program is started [7]. There are two settings in the input which are "smoothing kernel size (mm)" and "maximum hole size (mm)". Increasing the value for maximum hole size increases the threshold for what is considered a hole in the surface, meaning it prevents leakage into other cavities during the segmentation. It is unclear what Smoothing does - potentially just a smoothing kernel to reduce noise in the volume. To achieve the best possible results we followed the SlicerMorph team's guide on how to use SlicerMorph [19].

3.2.2 Testing SlicerMorph

Our input data was first preprocessed, as described briefly at the start of this section. We then tried to segment an endocranium based on this data, but also one where we used morphological operations and one where we cropped tightly around the endocranium - just like when testing Endomaker. Figure 7 shows an example of the cropped input volume rendered in 3D.

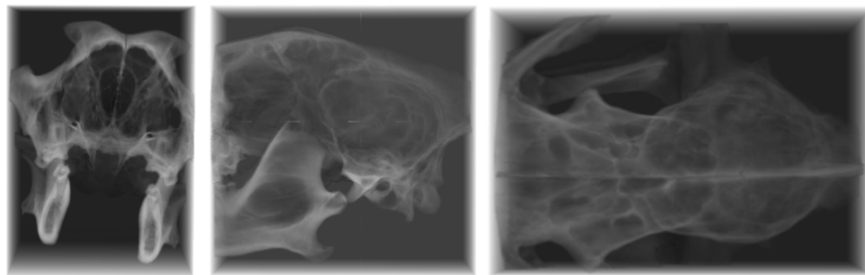


Figure 7: A rendering of a NIFTI-volume cropped tighter to the endocranium - seen from the front, side and top - that has resolution $\approx 0.48 \frac{\text{mm}}{\text{voxel}}$.

All of these examples have been tested on a consumer laptop that has 16 GB of ram and AMD Ryzen™ 5 7530U-processor where each run took a matter of minutes.

3.2.3 Results from SlicerMorph

SlicerMorph takes standardized volume file formats such as NIfTI. The whole process from loading the volume file, to the segmentation is done, only takes a few minutes given the input volume as mentioned and the laptop. Getting the correct output was quite challenging and required more effort than only inserting the preprocessed volume files. The main struggle being sensitivity in regards to input, or adjusting either of the two parameters. The Figures 8a and 8b show the two most common results we would get and how different they could be depending on the parameters. Using the cropped volume as input such that only one internal cavity is present, we got some decent results, as seen in Figures 8c and 8d. This brain segmentation, has no holes and is almost complete except for missing the olfactory bulb, as we also saw with Endomaker [9].

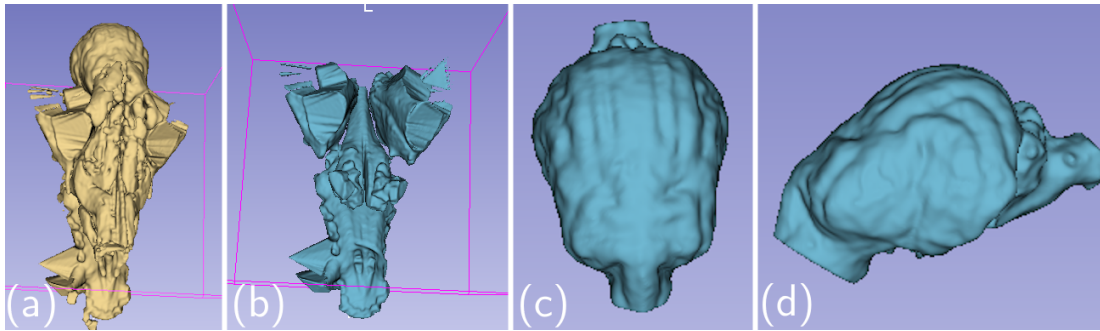


Figure 8: Different results from SlicerMorph. Exact settings for each can be found in appendices A.1, A.2 and A.3 for (a), (b), and (c) respectively. (d) is a side view of (c).

3.3 Discussion of and conclusion on existing methods

In general we found it quite challenging to get the current existing methods for digital endocast creation to work consistently - if even at all. We generally found there to be limited amount of information available about what different parameters actually do, and how they influence the result, making it difficult to tweak them to produce good results reliably. Even when we did get results representing the endocast, it would never be complete. The olfactory bulb was especially difficult for the methods to capture. SlicerMorph came close to what would actually be a clean visual result, but it was not complete.

We found that the current methods we tested did not seem viable for analyzing large amounts of μ CT-scans due to their sensitivity to the input and the need to tweak parameters blindly for each μ CT-scan volume. Even small tweaks in these parameters would greatly influence the results obtained.

Endomaker often produced error codes, as mentioned earlier. These were difficult to interpret and act upon, making it unreliable, as we were unable to identify what was the cause of them.

The two methods we tested were able to produce results on a $500 \times 500 \times 500$ voxel volume within a matter of minutes, which is good, though if you factor in the time needed to preprocess the volumes, this increases.

It is clear to us that more work can be done in this field to propose a new method for digital endocast creation from μ CT-scans. Preferably one that is easy to interact with and understand without needing a vast amount of knowledge about it, and one that is able to produce visually feasible and pleasant results, capturing the entirety of the endocranium.

4 Relevant Theory

This section covers relevant theory that will be applied in our proposed novel method for digital brain endocasting. It can therefore be of use for the reader to familiarize themselves with any topics in here that they do not know.

4.1 Downsampling

The resolution of the volume directly decides how "high resolution", or essentially how detailed, the endocast extracted using our proposed method is. Our method takes a NIfTI volume as its input. The resolution of a NIfTI volume indicates how much distance in world space, given in millimeters, is represented by a single voxel step along each of the three axes. Thus, this is directly linked to the resolution of the μ CT-scanner used to obtain the volume before it is converted to the NIfTI format.

The run time of our method, as well as the amount of memory needed to run it, heavily depends on the size of the volume, meaning the number of voxels in it. Therefore, it can be beneficial to somehow reduce the number of voxels, and thus the resolution, of the volume before attempting to apply our proposed method for brain endocast extraction if vast amounts of computing resources are not available or time is an issue.

This is done by downsampling the NIfTI volume. First a downsampling factor is chosen, in this chapter we denote it ρ . This factor determines the proportion of voxels kept along each axis after downsampling. For example, a factor of 0.25 results in only 25% as many voxels along each axis compared to the original volume. Usually, the downsampling factor will be the same for all three axes. The size of the downsampled volume, meaning also the amount of disk space it occupies, will be $\rho^3 \cdot 100\%$ of the original volume. So, if a downsampling factor of 0.25 is applied along all three axes of a volume, the resulting downsampled volume would be $0.25^3 \cdot 100\% \approx 1.56\%$ of the original volume's size in terms of voxel count.

We've utilized SciPy's `ndimage.zoom` function for downsampling of volumes. Given an original volume, $V_{orig} \in \mathbb{R}^{N_x \times N_y \times N_z}$, and a downsampling factor ρ , the function targets a new volume $V_{new} \in \mathbb{R}^{\lfloor \rho \cdot N_x \rfloor \times \lfloor \rho \cdot N_y \rfloor \times \lfloor \rho \cdot N_z \rfloor}$.

For each index:

$$i_{out_k} = \{0, 1, 2, \dots, \lfloor \rho \cdot k \rfloor - 1\}, \quad \forall k = \{N_x, N_y, N_z\} \quad (1)$$

in V_{new} along each of the axes, a coordinate in V_{orig} is chosen by:

$$k_{in_i} = \frac{i_{out_k} + 0.5}{\rho} - 0.5. \quad (2)$$

As the volume has the same number of voxels along each axis, the coordinates k_{in_i} will be the same for every axis. These new sample coordinates will not be actual indices in V_{orig} , as the 0.5 is added to them. Therefore, interpolation is needed. We use trilinear interpolation here and for multiple other steps later on in our proposed digital endocasting method where volume intensities are needed for points not in the original μ CT-scan volume [20].

By reducing the number of voxels along each axis, we also reduce the resolution of the resulting NIfTI volume, e.g. the distance in world space each single voxel step, along each of the three axes, represent. If the original volume had a resolution of $[r_x, r_y, r_z]$, meaning each voxel represents a r_x mm $\times$ r_y mm $\times$ r_z mm cube in physical space, the resulting resolution of downsampling, with a factor ρ , would be $[\frac{r_x}{\rho}, \frac{r_y}{\rho}, \frac{r_z}{\rho}]$.

4.2 Detection of a smooth layer in an image

Being able to detect and extract the first smooth layer in a gray-scale image is of great importance for our proposed method, as it will help detect the inside surface of the skull in the endocranium.

This section will describe how this is done in general with an example image for the purpose of illustration. The idea is heavily based on the one presented in [8].

Let $\mathbf{I} \in \mathbb{R}^{M \times N}$ be a gray-scale image of dimensions M by N containing a layer, meaning a continuous, relatively brighter structure in the image that extends across the width of the image and may have curvature. An example of such an image can be seen in Figure 9.

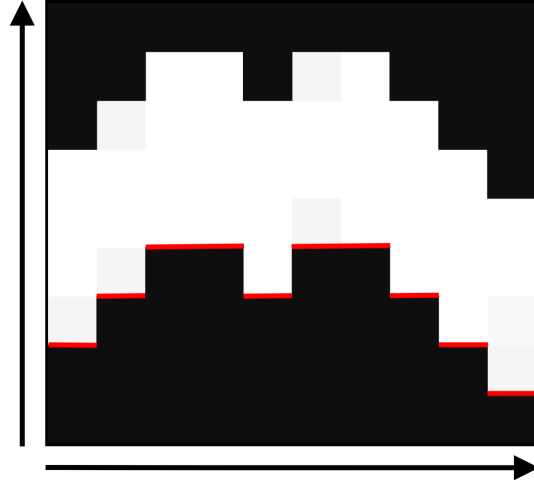


Figure 9: An example of an image $\mathbf{I}$ containing a layer. The index $[0,0]$ is at the lower left corner, with i increasing upwards and j increasing to the right. The red lines indicate the transition we want to locate, by grouping pixels above and below into their own sets.

Finding the lower boundary of the layer can be treated as a segmentation problem with the constraints that the segmentation must span the entire width of the image, and the segmentation should be relatively smooth. A graph based approach to performing this segmentation is appealing, as a graph can be constructed in such a way that a minimum cut on the graph yields a segmentation with these desired properties. The construction of this graph will now be described.

4.2.1 Construction of the graph

We now describe the construction of a flow network (graph) $\mathcal{G} = (V, E, s, t)$ consisting of a vertex set V , a set of directed edges E with capacities decided by $c : E \rightarrow \mathbb{R}_{\geq 0} \cup \{\infty\}$, a designated source vertex s and a designated terminal vertex t . We denote an edge from v to v' as $\langle v, v' \rangle$.

The vertices in V represent the transitions between adjacent pixels of the image $\mathbf{I}$ along each column. That is, for every pair of vertically neighboring pixels $\mathbf{I}[i, j]$ and $\mathbf{I}[i + 1, j]$, a vertex is placed to represent the transition between them. We write this vertex set as:

$$V = \{ v(i, j) \}. \quad (3)$$

We assume $i \in \{0, 1, \dots, M - 2\}$ and $j \in \{0, 1, \dots, N - 1\}$, unless otherwise stated, where M and N are the dimensions of $\mathbf{I}$. An illustration of these vertices, and how they correspond to $\mathbf{I}$, can be seen in Figure 10.

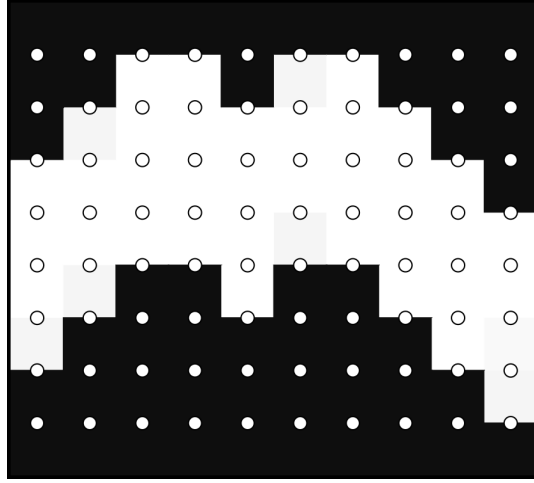


Figure 10: An example of the image $\mathbf{I}$ with vertices placed at pixel transitions along the columns.

We wish to be able to locate the first, meaning the lowest index, clear transition into the layer along each column using a minimum s - t cut. To have a minimum cut, we need to have a flow on the graph, meaning we need s and t to somehow be connected to the vertices in V . We therefore introduce what we call the set of *external edges*, containing edges going from s to all $v(i, j)$ and from all $v(i, j)$ to t . Formally:

$$E_E = \{ \langle s, v(i, j) \rangle, \langle v(i, j), t \rangle \}. \quad (4)$$

An illustration of the external edges in relation to the vertices in V can be seen in Figure 11.

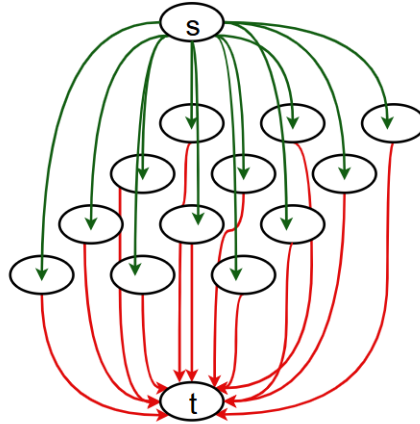


Figure 11: An example of external edges, where green edges go from the source s into the vertices in V and red edges go from the vertices in V to the terminal t .

These external edges will help define the segmentation of the vertices in V . In the end, every vertex $v(i, j)$ will either have the edge going from s to it or the edge going from it to t be in the minimum cut, as a result of how the remaining part of the graph is constructed. Let E_{cut}

be the set of edges in E that make up a minimum s - t cut. We define the sets of foreground and background vertices, V_f and V_b respectively, as:

$$\begin{aligned} V_f &= \{ v \in V \mid \langle s, v \rangle \in E_{cut} \}, \\ V_b &= \{ v \in V \mid \langle v, t \rangle \in E_{cut} \}. \end{aligned} \quad (5)$$

We wish to find only one layer in the image. Therefore, the vertices of each column are connected by directed edges pointing upwards:

$$E_C = \{ \langle v(i, j), v(i+1, j) \rangle \mid i \neq M-2 \} \quad (6)$$

These edges all have the capacity:

$$c(\langle v, v' \rangle) = \infty, \quad \text{if } \langle v, v' \rangle \in E_C, \quad (7)$$

as this ensures that none of these edges can be in a minimum s - t cut. This directly enforces that

$$v(i, j) \in V_b \Rightarrow v(i', j) \in V_b, \quad \forall i' > i, \quad (8)$$

meaning that once a vertex in a column is in the background, then so are all vertices above it, effectively resulting in only a single layer being found.

We wish that the layer that is found is relatively smooth, meaning that the transition from foreground to background should not be far apart in neighboring columns. In this instance, we define the $\text{neighbor}(j, j')$, which takes in two column-indices and returns:

$$\text{neighbor}(j, j') = \begin{cases} 1 & \text{if } j = j' \pm 1 \\ 0 & \text{if } j \neq j' \pm 1, \end{cases} \quad (9)$$

meaning it outputs 1 if the two columns are directly next to each other.

We want to be able to control it so that the found layer can only shift a maximum of $\Delta i \in \mathbb{N}^+$ in its i -index between neighboring columns.

To do this, we add a set of slanted inter-column edges between vertices in neighboring columns in the following way:

$$E_S = \{ \langle v(i, j), v(i + \Delta i, j') \rangle \mid i \leq M-2-\Delta i \wedge \text{neighbor}(j, j') = 1 \}. \quad (10)$$

These edges all have the capacity:

$$c(\langle v, v' \rangle) = \infty, \quad \text{if } \langle v, v' \rangle \in E_S, \quad (11)$$

as this once again ensures that none of these edges can be in a minimum s - t cut. This directly enforces that:

$$v(i, j) \in V_b \Rightarrow v(i + \Delta i, j'), \quad \forall j' \text{ s.t. } \text{neighbor}(j, j') = 1, \quad (12)$$

meaning that the start of the background in two neighboring columns cannot be more than Δi apart in the row-index.

We will be referring to the set of edges

$$E_I = E_C \cup E_S, \quad (13)$$

defining the grid-like structure, as the *internal edges* of $\mathcal{G}$. An illustration of the vertices and internal edges on top of the example image can be seen in Figure 12.

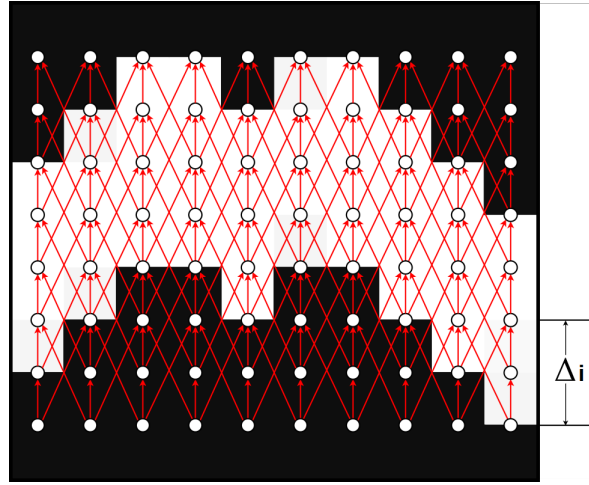


Figure 12: The example image $\mathbf{I}$ with its vertices and internal edges shown on top. Here $\Delta i = 2$.

We now have the entire set of edges of the graph:

$$E = E_I \cup E_E. \quad (14)$$

To now define the capacities assigned to the external edges, that will help ensure we find the first layer in the image, we introduce the matrix $\mathbf{A} \in \mathbb{R}^{(M-1) \times N}$, where each element in it is defined as

$$\mathbf{A}[i, j] = |\mathbf{I}[i, j] - \mathbf{I}[i + 1, j]|. \quad (15)$$

$\mathbf{A}[i, j]$ thus captures the absolute difference in intensity between vertically adjacent pixels in $\mathbf{I}$, essentially encoding the vertical gradient magnitude at the transition-point corresponding to $v(i, j)$. An illustration of an example $\mathbf{A}$ (corresponding to the prior examples) can be seen in Figure 13.

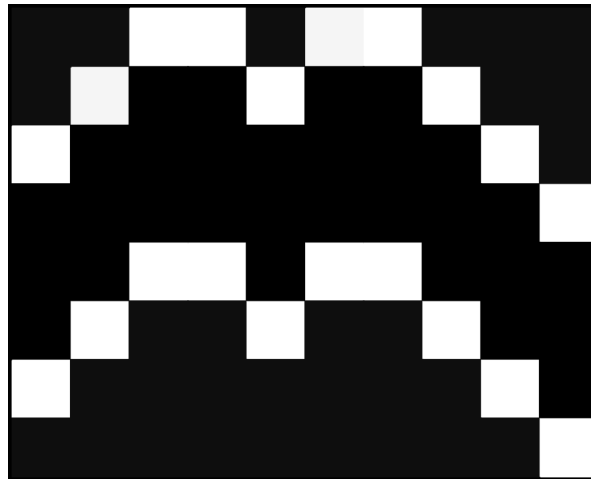


Figure 13: The gradient image corresponding to that of the image $\mathbf{I}$.

Let $v_0 = v(i_0, j_0)$ be an arbitrary vertex. We define the function $\mathbf{A}(v_0)$ to return the corresponding matrix value:

$$\mathbf{A}(v_0) := \mathbf{A}[i_0, j_0]. \quad (16)$$

This function thus maps a vertex v_0 to its associated image gradient.

We want the vertices that lie between pixels of high contrast in $\mathbf{I}$, meaning vertices with corresponding high values in $\mathbf{A}$, to be segmented into the foreground set V_f . Therefore, we now define the capacity function on the external edges as well, so that we have a complete capacity function:

$$c(\langle v, v' \rangle) = \begin{cases} \infty & \text{if } \langle v, v' \rangle \in E_I \\ (M-1) \cdot \mathbf{A}(v') & \text{if } v = s \\ \max_{i,j} \mathbf{I}[i, j] - \mathbf{A}(v) & \text{if } v' = t. \end{cases} \quad (17)$$

This capacity function yields very high capacities on edges $\langle s, v(i, j) \rangle$ when $\mathbf{A}(i, j)$ has a high value, meaning when the absolute value of the image gradient is high, and low values when this value is low. For edges $\langle v(i, j), t \rangle$ it yields quite high values (close to the maximum value in $\mathbf{I}$) when the absolute value of the gradient is low, and down to low values when it is very high.

4.2.2 Properties of a minimum s - t cut on the graph

This definition of the capacities helps ensure that all vertices $v(i, j)$ that have high corresponding absolute values of gradients, do not have their $\langle s, v(i, j) \rangle$ -edge in the minimum cut, as it would be too 'expensive', meaning they are put in the background set V_b . This is what ensures we find the first layer in the image.

When this is combined with the internal edges, a minimum s - t cut in the graph will result in a segmentation that finds a best-fit first and smooth layer in the image. How the minimum cut defines the segmentation can be seen in Figure 14.

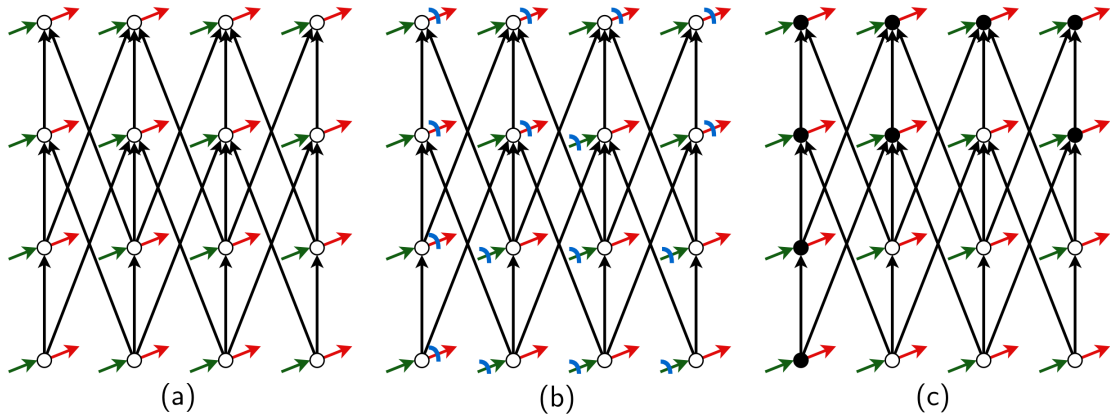


Figure 14: (a) shows a graph constructed in the way described earlier. (b) shows a cut on this graph, indicated by the blue lines. (c) illustrates the resulting segmentation of the vertices of the graph induced by the cut in (b).

Finding a minimum cut on the used example image and segmenting the vertices into foreground and background yields the result seen in Figure 15 on the next page.

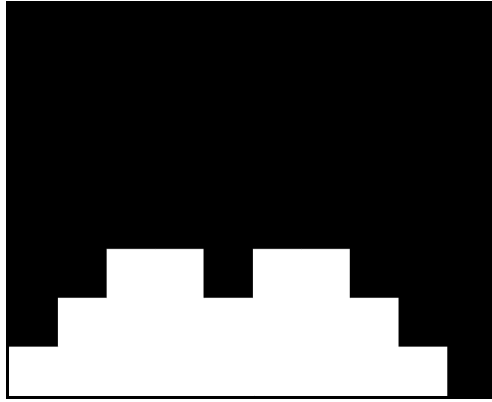


Figure 15: The obtained segmentation of the vertices (now appearing as pixels).

For each column in the image $\mathbf{I}$, if $v(i, j) \in V_f$ and $v(i + 1, j) \in V_b$, then the layer will be at pixel $\mathbf{I}[i + 1, j]$ in the image. Meaning that the layer in the image lies in the transition from foreground to background in the segmented graph.

5 Our novel method

We now introduce our proposed method for creating digital brain endocasts from μ CT-scans of skulls. From a high-level perspective, our method works in the following way on a supplied μ CT-scan $\mathbf{V} \in \mathbb{R}^{N_x \times N_y \times N_z}$ containing a skull:

1. The bone structure in the volume, meaning the skull itself, is located. This is described in section 5.1.
2. The volume is rotated such that the standard anatomical axes are aligned with the axes of the volume. This is described in section 5.2.
3. An estimation of the center point of the endocranium of the skull is calculated. This is described in section 5.3.
4. At this center point a spherical mesh is initialized inside of the endocranium. This is described in section 5.4.1.
5. At last, the mesh will be expanded iteratively outwards into the endocranium of the skull, eventually creating an endocast of the skull. This is described in section 5.4.3.

This pipeline can be seen illustrated in Figure 16 with corresponding numbering for the steps described above.

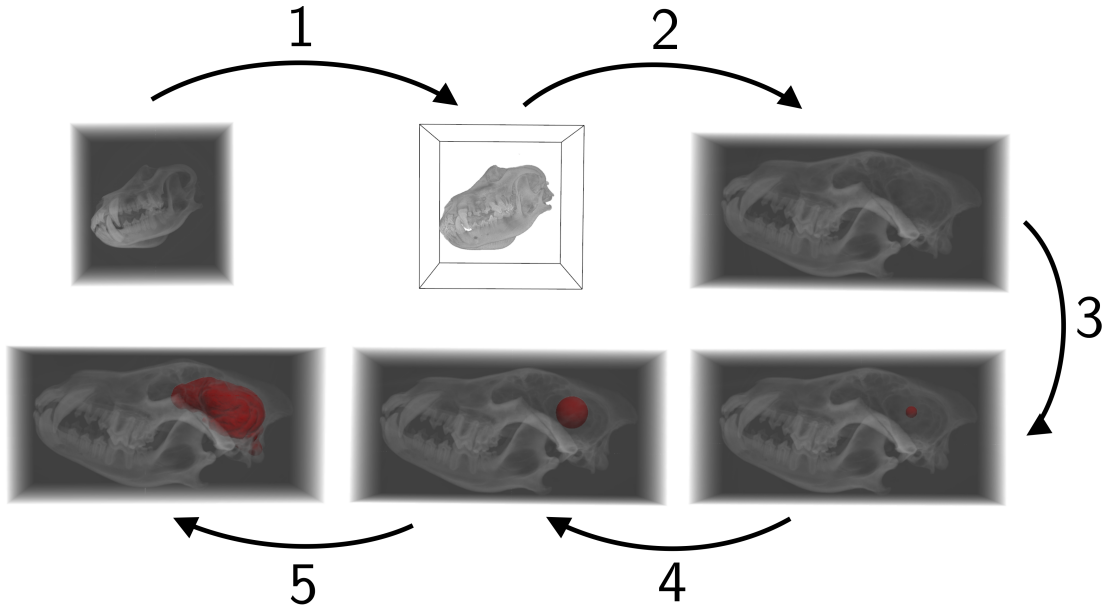


Figure 16: An illustration of the pipeline in our novel method for digital brain endocasting. 1) Locate skull in volume. 2) PCA align to anatomical axes. 3) Find estimated center point. 4) Initialize mesh icosphere. 5) Expand mesh iteratively.

5.1 Segmenting bone structure of interest in volume

5.1.1 Locating bone

Given a NIfTI volume representing a μ CT-scan of a skull, we first want to identify where in the volume there is bone, and where there is not. To do this, we use Otsu's Method for thresholding. The voxel grid with intensities is loaded in, after which Otsu's Method yields a specific voxel intensity threshold that separates the voxels into two classes; foreground and background. The

intensity threshold is determined by maximizing inter-class variance [21]. The found threshold is then applied to the volume, resulting in a binary volume where "1" represents bone and "0" represents non-bone.

Otsu's Method works well when the voxel intensity histogram has a clear bimodal distribution with an as deep and sharp valley as possible between the two peaks. This fits well with μ CT-scans of skulls as bone is quite clearly defined in these. Figure 17 shows a typical voxel intensity distribution of a skull μ CT-scan. Note however, that the number of non-bone voxels greatly outnumbers the number of bone voxels, which is why a log-scale y-axis is used.

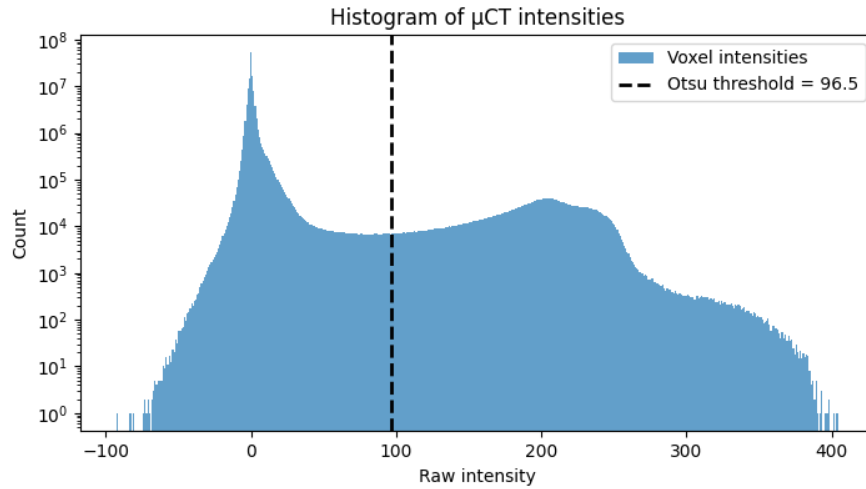


Figure 17: An approximate histogram of voxel intensities for a wolf μ CT-scan with the dotted line indicating the found threshold using Otsu's Method.

5.1.2 Finding the largest contiguous bone-structure

In the new binary volume a voxel intensity of "1" should indicate bone and one of "0" should indicate non-bone. The output of a threshold segmentation is, however, rarely perfect. μ CT-scans always have some degree of noise, there can be potential imaging artifacts or additional objects in the scan. The binary volume therefore is also prone to this noise after thresholding. To combat this, BLOB analysis is used. BLOB stands for binary large object, and refers to a group of pixels that are connected and therefore together make up an object in a binary image. In our method, SciPy is used for BLOB analysis. It uses a spherical connectivity, meaning a foreground voxel is connected to any other foreground voxel that is one of its 6 direct neighbors [22].

First, all BLOBs in the binary bone volume are found. These are all of the connected components in the binary volume. The only element that is strictly needed to generate an endocast is the neurocranium, which is the part of the skull enclosing the cranial cavity [23]. This will typically either be the largest connected component or in it. Therefore, all but the largest BLOB (the BLOB with the most total voxels in it) are discarded, meaning their intensities are turned to "0". This results in only the largest contiguous bone structure being retained in the binary volume. This removes noise and perhaps other large bones such as the jaw, which are not of interest anyway. There may, however, exist animals with different skull proportions for which this assumption of the neurocranium being the largest connected component does not hold. But, in the case of wolves, it seems reasonable.

After this step we have a relatively noise-free binary volume indicating the largest contiguous bone in the original μ CT-scan.

5.2 Creating an axis-aligned volume

5.2.1 Rotation of the volume

For practical reasons and for imaging quality, the skull is not necessarily placed such that the natural anatomical axes (front-back, up-down, left-right) align with the axes (x, y, z) of the reconstructed μ CT-scan NIfTI volume $\mathbf{V}$. Having these axes aligned can be of benefit to the user when visualizing the volume by slicing through it, or for purposes such as applying heuristics to locate points in certain areas of the skull.

This alignment can be achieved using PCA on the skull [24]. The binary volume containing the largest contiguous bone is a direct representation of the region of interest in the skull, or perhaps the entire skull, if it is connected. The indices in which this volume has 1's thus make up the point coordinates of the skull in an index-space. We call this set of 3D coordinates $\mathbf{P}_V \in \mathbb{R}^{N \times 3}$, where N is the number of 1-indices in the binary skull volume. These points can be transformed into world coordinates by applying the resolution information stored in the affine matrix of the NIfTI volume. Let $\mathbf{s} = [s_x \ s_y \ s_z]^T$ be the vector containing the voxel resolution along each axis in $\mathbf{V}$. Then, the skull points in world coordinates are given by $\mathbf{P}_W = \mathbf{P}_V \mathbf{s} \in \mathbb{R}^{N \times 3}$. This is especially important for the PCA-analysis if the volume is not isotropic.

Applying PCA to the point cloud $\mathbf{P}_W$ allows it to be rotated such that its principal components align with the coordinate axes. A subset of the entire point set is used to increase speed while still performing just as well, we use a subset of 5000 points. The three principal components with the highest explained variance will in a vast majority of cases align well with the standard anatomical axes. Which principal component corresponds to length, height, and width depends on the anatomical properties of the skull.

It can be beneficial for comparative analysis of skulls, or in the end endocasts, to align them using a consistent PCA-based convention, where principal directions correspond to fixed anatomical axes. We've chosen the following convention for the three principal components:

- The positive direction of the first principal component (PC1) points from the back to the front of the skull.
- The positive direction of the second principal component (PC2) points from the bottom to the top of the skull.
- The positive direction of the third principal component (PC3) points from the left to the right side of the skull.

An automatic algorithm to perform this is beyond the scope of this project. Instead, we rely on user interaction through a simple graphical user interface (GUI) that the user is prompted with. The user can then in an easy and interactive manner swap principal components or change their directions to make sure it aligns with the convention above. A screenshot of this GUI can be seen in Figure 18 on the next page. It also shows that the skull is much more well-aligned in PCA-space.

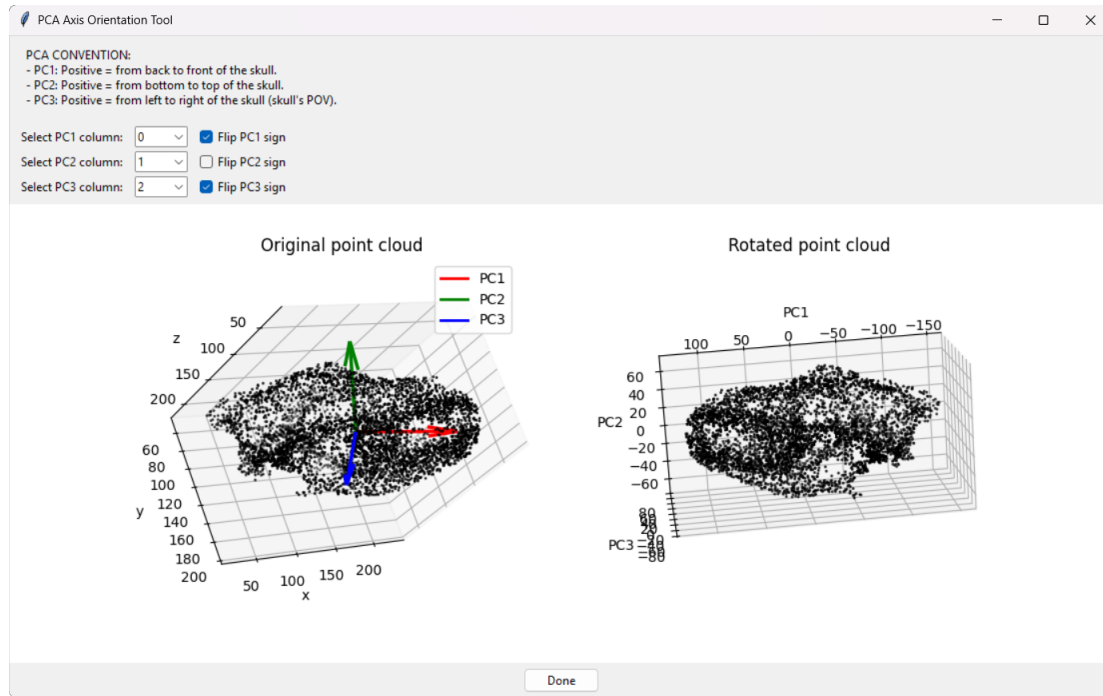


Figure 18: PCA GUI tested on a wolf μ CT-scan. Manual adjustments have been made so that PC1 and PC2 have had their directions flipped to align with our convention.

5.2.2 Resampling of the volume

The final PCA transformation found by the user in the GUI can now be used to create a PCA-aligned version of the original NIfTI volume. In this volume the standard anatomical axes would directly correspond to the axes of the volume in the way defined by our convention.

A bounding box of the subset of points from the skull point cloud mentioned in section 5.2.1 is found in PCA-space. Then, a padding of 5% of the bounding box's size is added on each side of the bounding box for each PC-axis as a precaution to ensure nothing is missed. This new box with the padding essentially encapsulates the new PCA-aligned volume. A regular grid with the same voxel spacing as the original volume $\mathbf{V}$ is set up inside of this box. This yields a new set of points in PCA-space that make up a volume in which the skull lies. These points do not necessarily, in fact they will most often not, correspond to indices existing in the original NIfTI volume. Therefore, interpolation of the intensity at these points in the original volume is needed. First, the inverse PCA transformation is applied to all of the points to find the corresponding points in the original volume. Then, the intensity at each point is calculated using trilinear interpolation [20]. When this is done, a new volume $\mathbf{V}_{PCA}$ is obtained.

The resolution of the original NIfTI volume $\mathbf{V}$ is preserved and so the new PCA-resampled NIfTI volume $\mathbf{V}_{PCA}$ has the same resolution. This PCA-resampling can potentially reduce the size of the volume by limiting the region of interest through the bounding box, which might make up only a small amount of the total volume. This is yet another potential benefit of doing this step.

5.3 Estimating the center point of the endocranium

5.3.1 Using the convex hull to limit the search space

We can greatly reduce the space in which we need to perform our search for the center point of the endocranium by using the information we have already found about the bone structure in the now axis-aligned volume. By applying the steps from section 5.1 on the new axis-aligned volume $\mathbf{V}_{PCA}$, we can once again identify the relevant bone structure in the new volume. The points $\mathbf{P} \in \mathbb{R}^{N \times 3}$ that are found to be bone in $\mathbf{V}_{PCA}$ make up a point cloud. We know that the center point of the endocranium must lie inside of this point cloud. Therefore, we utilize three-dimensional Delaunay triangulation to generate the mesh of a convex hull encapsulating the points in $\mathbf{P}$, meaning the surface of the skull [25]. This generates a smaller region of interest (ROI) for our search. We subsample a set of 5000 points from $\mathbf{P}$ and use **PyVista** to perform the Delaunay triangulation [13]. This constructs the Delaunay triangulation of the point cloud whose surface triangular faces make up a mesh that encapsulates the point cloud.

In Figure 19 an example of a convex hull mesh can be seen from three different angles.

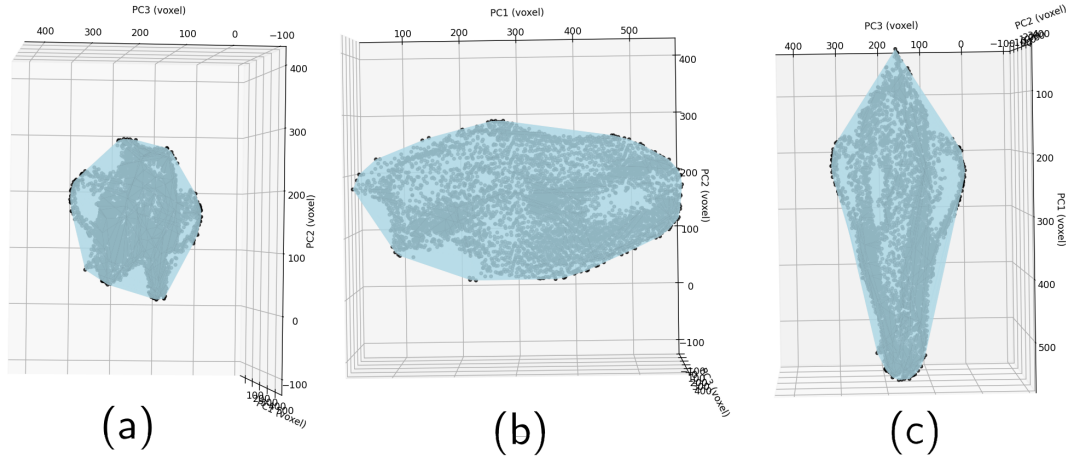


Figure 19: The convex hull of a wolf μ CT-scan seen from the front (a), the side (b), and the top (c). The mesh fully encapsulates the point cloud, the reason it may not appear like it is due to the rendering size of the points.

Once a convex hull mesh is obtained, a new binary volume is created which has the same dimensions as $\mathbf{V}_{PCA}$ with voxels of value "1" indicating that they are inside of the convex hull and voxels with a value of "0" indicating that they are outside of the convex hull. This is done using `vtkPolyDataToImageStencil`, which looks at one axial slice after another, projecting the mesh outline onto that 2D plane and doing a simple point-in-polygon fill to mark interior pixels. This ROI will be used in the following section to help limit the number of candidate points for the center of the endocranium.

5.3.2 Finding candidate center points and picking the right one

The endocranium is a relatively large - at least in the case of wolves species - cavity inside of the skull. It is, perhaps, the largest. Therefore, we can assume the following about the center point of the endocranium:

- It lies within the found ROI defined in section 5.3.1.
- It has the largest distance to the nearest bone in its local neighborhood in the volume.

Therefore, if we can locate such points, we will have a relatively restricted set of points one of which will be a good estimation of the center point of the endocranium.

We have a binary volume in which "1" represents bone and "0" represents air. By inverting this, such that "1" represents air and "0" represents bone, we can now apply a distance transform to this new volume. A distance transform transforms a binary volume by replacing every foreground voxel (i.e. voxels with a value of "1") with their distance to the nearest background voxel (i.e. voxels with a value of "0") [22]. Doing this generates a new volume in which voxels with a high intensity are far from the nearest bone structure.

In this volume, we wish to find the local maximum values, meaning for a specific neighborhood size, we want to find the voxel that has the highest value. The distance transformed volume can be quite noisy, as it is sensitive to even small changes in the original volume, making locating a local maximum more difficult. Therefore, we apply a gaussian smoothing filter on this volume, with $\sigma = 3$, in order to smoothen it out, making the local maximum values more clear.

We use Scikit-Image's `peak_local_max` function to then locate the local maximum values in the smoothed distance transformed volume [17]. It finds local maxima in a region of $2 \cdot \text{min_distance} + 1$, meaning that the found local maxima are separated by at least `min_distance`. This `min_distance` can be adjusted to restrict the amount of local maxima found. If one has a good estimation of the dimensions of the endocranium this can be used to set the neighborhood size accordingly. We have used `min_distance = 50`.

Figure 20 shows the steps going from a binary mask of the skull to the found local maxima points. Note, that these images are just 2D representations of what happens in the 3D volume. A mesh to define the ROI has therefore not been used to reduce the search space.

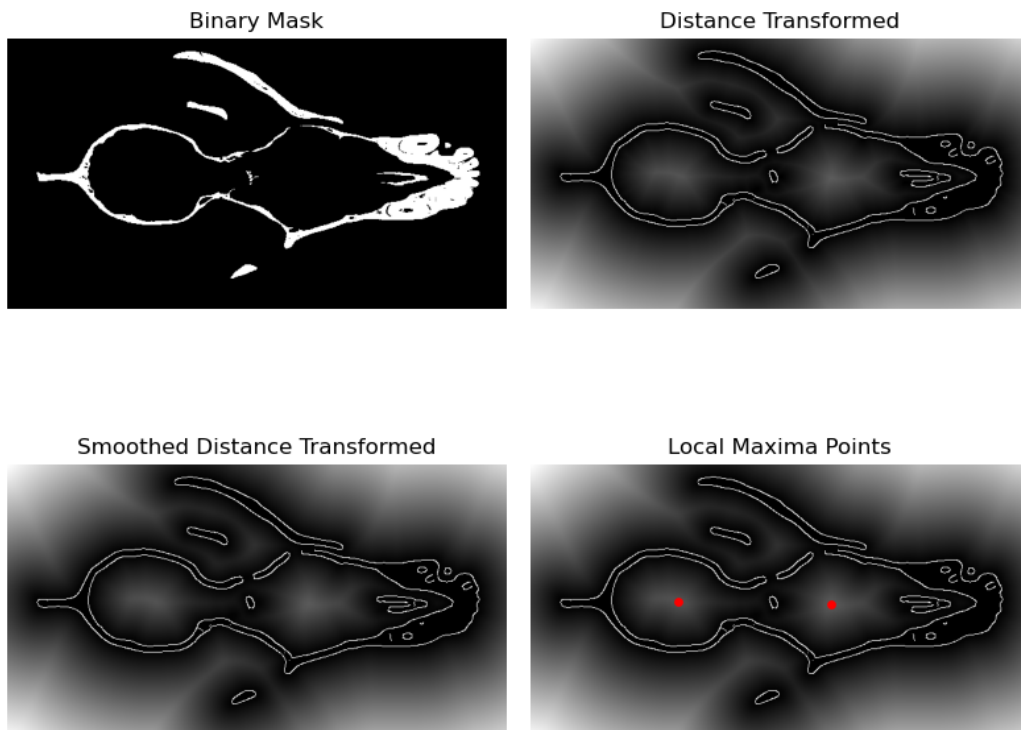


Figure 20: The steps of going from a binary skull volume to finding the local maxima points in a 2D example. The edges from the binary mask are overlaid on the following images solely to enhance visual clarity and are included only for illustrative purposes.

In the end, we can use the volume mask defining the voxels inside of the ROI to only look at found local maxima points that are inside of this mask to reduce the number of candidate endocranium center points.

Making this process completely automatic is outside the scope of this project. We have therefore implemented a simple GUI to allow the user to choose the endocranium center point among the found local maxima points. The user gets to cycle through them, starting with the found local maxima with the furthest distance to its nearest bone, and providing them in decreasing order. This GUI can be seen in Figure 21.

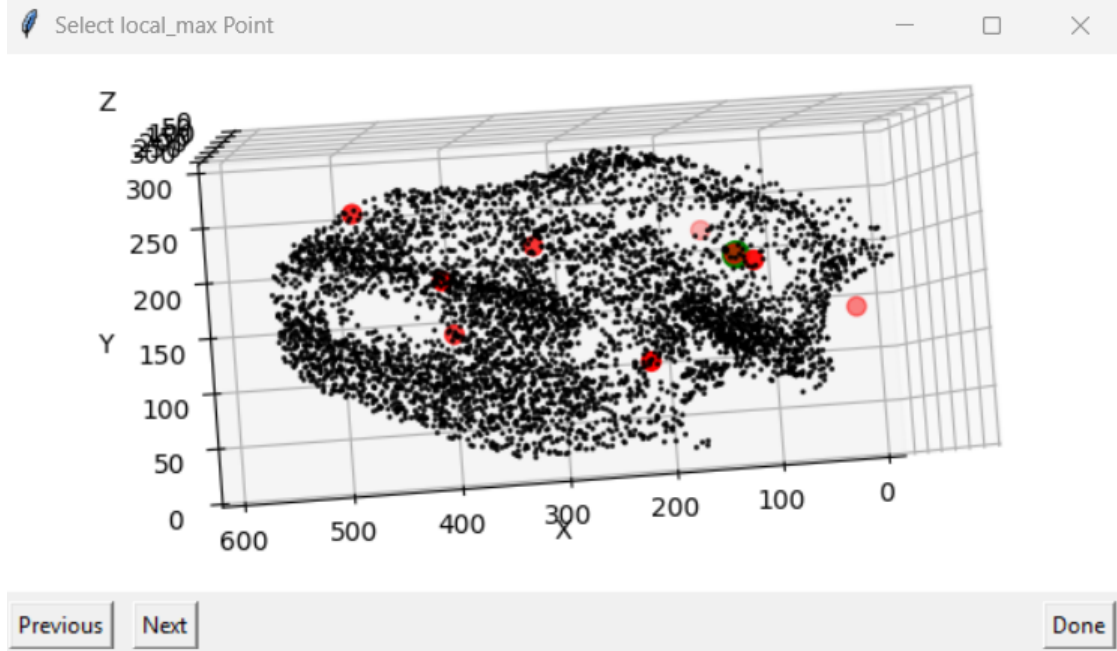


Figure 21: Our implemented GUI for choosing the center point of the endocranium shown on a 0.25 times downsampled wolf skull. The user can cycle through the points using the buttons at the bottom.

The point chosen by the user has a certain distance to the nearest bone in the volume, we call this distance d_{min} , which will be used in section 5.4.1.

5.4 Generating the endocast

At this point, we have:

- A NIfTI volume $\mathbf{V} \in \mathbb{R}^{N_x \times N_y \times N_z}$ (which should be axis-aligned).
- The voxel coordinates in the NIfTI volume of a candidate center point of the endocranium, $p = [x_0 \ y_0 \ z_0]^T$.
- A minimum distance, d_{min} from section 5.3.2, from p to the skull structure in $\mathbf{V}$.

This section will dive into how a spherical mesh is created at the center point in the endocranium, and then expanded outwards iteratively to fill out the endocranium and generate an endocast of the skull.

On a high-level, the following will be done: A spherical mesh is initialized at p with as large of a radius as possible, without the sphere touching the inside of the skull. Then, the normals of the vertices in the mesh are calculated, and sample rays are shot outwards along these. The

voxel-intensities at intermediate steps along these normal vectors are calculated by interpolation in the volume. From these sample points a graph, whose minimum cut has desirable properties, is constructed and a minimum cut is found. This gives an estimation of the point along each normal vector where the inside of the skull is hit, subject to a constraint that ensures that these points should not lie far from each other for the normals of mesh vertices that are neighbors. The mesh vertices are then shifted along these normals to the found point, after which the new mesh is smoothed. From here, new vertex normal rays are shot out, and the same procedure as before continues iteratively for a set amount of iterations with only minor changes from the first iteration. At some steps the mesh is subdivided to improve its ability to capture details in the endocranium.

5.4.1 Preparing a spherical mesh at the center of the endocranium

First, a triangulated spherical mesh has to be created at the center point of the endocranium p . We use what is called an *icosphere*, which is a geodesic icosahedron with subdivision frequency ν , as proposed by Dahl et al. [10]. For a given ν , each edge of the icosahedron will be split into ν segments, and each face will be split into ν^2 faces. This allows for a high degree of control over the mesh resolution of the initial sphere, as the number of vertices grows as

$$n_{verts} = 12 + 10 \cdot (\nu^2 - 1), \quad (18)$$

which is quadratic. Our method has $\nu = 10$ as its standard, meaning the initial icosphere centered at p has 1002 vertices. Examples of icospheres can be seen in Figure 22.

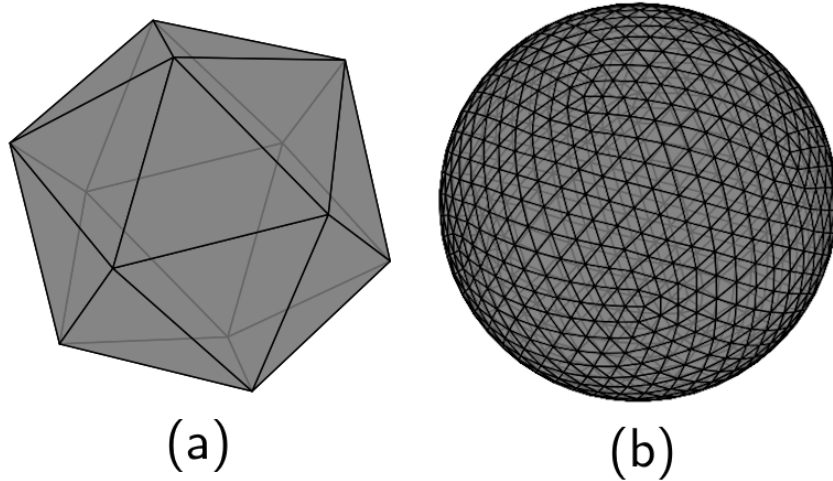


Figure 22: (a) An icosphere with $\nu = 1$ and (b) an icosphere with $\nu = 10$.

As we will be shooting rays out from the icosphere into the volume to hit bone, we want to limit the sampling distance needed out into the skull. We can reduce this by initializing the icosphere with as large a radius as possible, within reasonable and safe bounds. We know that the part of the skull that is closest to the center point p is at a distance of d_{min} from p . We therefore choose to initialize the icosphere with a radius of $\text{round}(0.8 \cdot d_{min})$. This leaves some extra space just as a precaution.

From the icosphere a neighbor list is created. This is done by circulating all vertices, meaning for each vertex the faces that it is a part of are found. The other vertices that are in these faces

are the *neighbors* of this vertex. We define the operator

$$\text{neighbor}(j, j') = \begin{cases} 1 & \text{if vertices } j \text{ and } j' \text{ are neighbors} \\ 0 & \text{if vertices } j \text{ and } j' \text{ are not neighbors.} \end{cases} \quad (19)$$

Now, for each vertex in the mesh its unit normal vector is calculated. This is just an (angle) weighted average of the surrounding face's normal vectors scaled to a unit vector [26]. We will be referring to these vertex unit normal vectors simply as the mesh normal vectors or the normal vectors.

5.4.2 Sampling along mesh normal vectors

We will be sampling steps along these normal vectors in the the NIfTI volume. As this can easily become computationally expensive if a very large amount of steps are taken, we wish to be able to reduce the number of steps taken out from the icosphere as much as possible. We do, however, preferably still want to reach the inside of the skull along as many of the rays as possible. To estimate an appropriate amount of initial steps, we make some assumptions about the size of the volume that the endocranium takes up. The NIfTI volume is assumed to be PCA-aligned, meaning the volume axes correspond to the standard anatomical axes.

We expect the endocranium to take up:

- a maximum of 60% of the length of the volume, N_x ,
- a maximum of the entire height of the volume, N_y ,
- and a maximum of the entire width of the volume, N_z .

We call the largest of the above l_{max} and the box of size $0.6N_x \times N_y \times N_z$ centered at p the *endocranium box*. Note that this assumption about the endocranium might not hold true for the skulls of all animal species, but we have found that in our tests cases it works well.

Figure 23 shows an example of a wolf skull with its corresponding endocranium box.

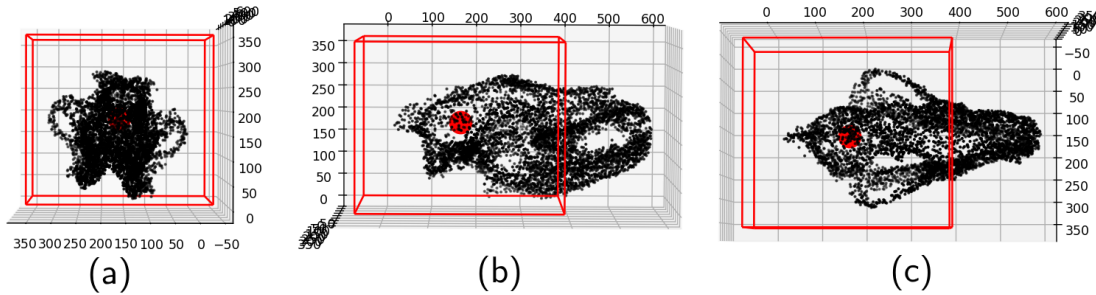


Figure 23: An illustration of a wolf skull (black point cloud) with its endocranium box and the center point of the endocranium. Seen from the front (a), from the side (b), and from the top (c).

With this assumption, we can now say that if we sample $\lfloor 0.5 \cdot l_{max} \rfloor$ unit vector steps in all directions from the center point p , we should hit the inside of the skull in all directions. This should hold even in a worst-case scenario, where the estimation of the endocranium center point is not very good. As a precaution, we sample $\lfloor 0.6 \cdot l_{max} \rfloor$ unit vector steps in all directions instead.

The standard for our method is to take steps of length 1, meaning going one unit normal vector out per step. Therefore, we should take

$$\lfloor 0.6 \cdot \max(0.6 \cdot N_x, N_y, N_z) \rfloor, \quad (20)$$

steps in all directions from a unit icosphere. We floor to ensure that the number of steps is integer. As we have that the radius of the icosphere is $r = \text{round}(0.8 \cdot d_{\min})$, we can remove that amount of steps, meaning that we need to take

$$s_0 = \lfloor 0.6 \cdot \max(0.6 \cdot N_x, N_y, N_z) \rfloor - \text{round}(0.8 \cdot d_{\min}), \quad (21)$$

steps in all directions from the icosphere.

We introduce the tensor $\mathbf{C} \in \mathbb{R}^{M \times N \times 3}$, where M is the number of steps taken plus one, here $s_0 + 1$, and N is the number of vertices in the mesh. It holds the coordinates of the point reached when traveling i steps along the normal vector of vertex j - at least in this case. We will later on allow sampling in both the opposite direction of the normal vector as well. We define

$$\mathbf{p}_{ij} = \begin{bmatrix} \mathbf{C}[i, j, 0] \\ \mathbf{C}[i, j, 1] \\ \mathbf{C}[i, j, 2] \end{bmatrix} \quad (22)$$

as the coordinates of the point reached when traveling i steps along the unit normal vector of vertex j . An illustration of a skull point cloud along with a set of sample coordinates can be seen in Figure 24.

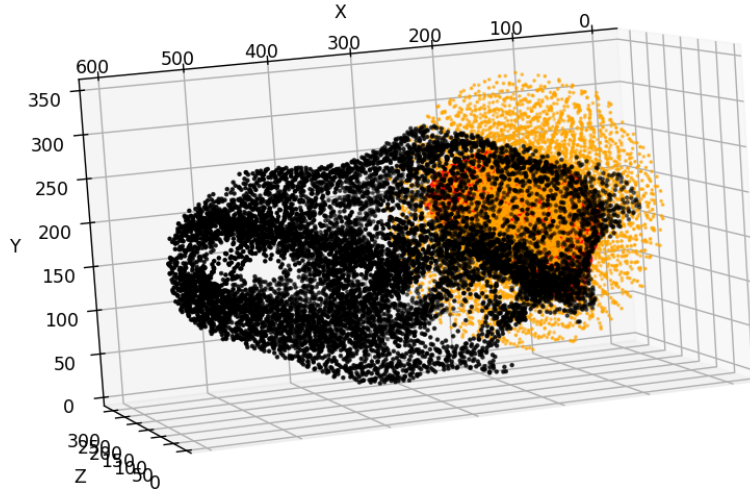


Figure 24: An illustration of a wolf skull (black point cloud) with a subset of the sample coordinates of a custom $\mathbf{C}$ (orange point cloud).

Now, we want to sample the voxel intensity at these newly found coordinates in the NIfTI volume. As done previously when resampling the NIfTI volume, trilinear interpolation is used to interpolate $\mathbf{V}$ between indices, creating the interpolated volume $\tilde{\mathbf{V}}(x, y, z) \in \mathbb{R}$, which is essentially a function. A new matrix, $\mathbf{I} \in \mathbb{R}^{M \times N}$, is created, where M and N correspond to those of $\mathbf{C}$. The entry at index $\mathbf{I}[i, j]$ is simply $\tilde{\mathbf{V}}(\mathbf{p}_{ij})$, meaning the interpolated voxel intensity at the point $\mathbf{p}_{ij}$. We therefore call $\mathbf{I}$ the *column-image*. An example of a column-image can be seen in Figure 25. It can be seen that there appears to be some sort of layer. Keep in mind that the neighbors of each column might not be the ones directly next to it, as they are defined from an icosphere mesh.

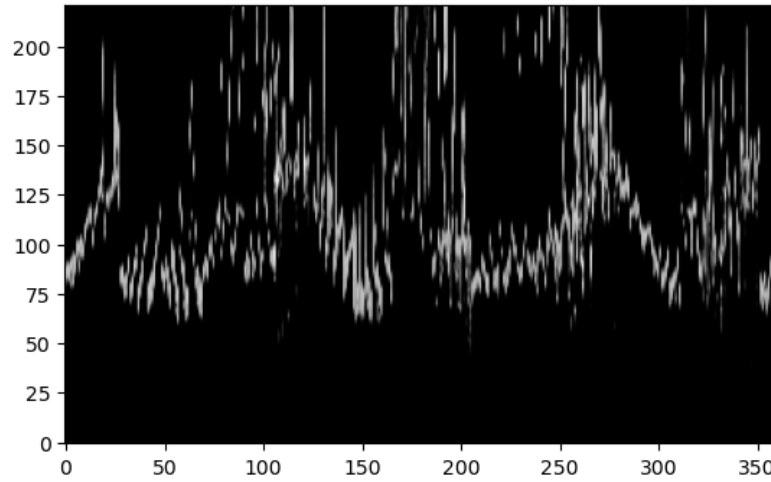


Figure 25: A column-image, with $M = 221$ and $N = 362$, from the initial sampling along the normals of the icosphere in a wolf μ CT-scan. The white voxels indicate bone structure that the rays have hit.

In this way, we have now sampled along all of the vertex normals of the icosphere in the NIfTI volume, where each row in the column-image corresponds to one layer of moving outwards, so to say.

5.4.3 Moving mesh vertices closer to the inside surface of the neurocranium

The obtained column image $\mathbf{I}$ contains information about what is encountered in the volume when moving along the mesh vertex normal vectors. As seen in Figure 25, there appears to be some sort of 'layer' in the column image. This layer indicates when bone is encountered along each normal vector. The layer may not seem very well-defined, but this is because of the way neighboring columns are defined. In this case, every column has multiple neighbors, as the columns correspond to mesh vertices.

This layer indicates the step along each normal vector that hits the inside of the skull. This is thus the point we wish to move the vertex to, in order to create the endocast of the skull. We therefore need to be able to extract this layer from the column image.

We can utilize the graph cut segmentation method described in section 4.2 to detect the first smooth layer in the column image. We need to make minor tweaks to this method in order for it to work as intended in our case where the concept of neighboring is different from the one described in that section.

In the case of the column image generated from a mesh, we use the neighbor operator defined in (19) for meshes, rather than the one defined in section 4.2 at (9) for simple 2D images. Figure 26 on the next page shows how a vertex is connected to its neighbors when using this new way of defining neighboring.

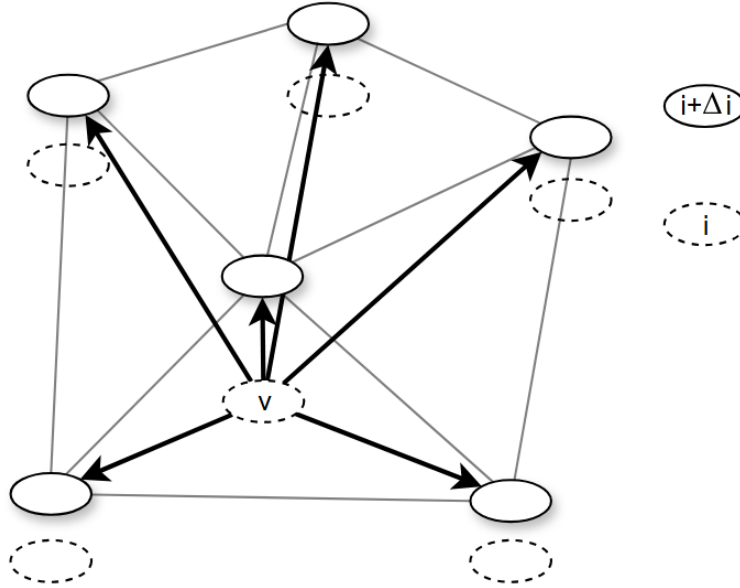


Figure 26: An illustration of the mesh structure and vertex v 's outgoing edges. Here $\Delta i = 1$. The arrows indicate directed edges in a graph. The thin lines help indicate the mesh triangular faces.

With this change we now ensure that if a vertex in the mesh has to be moved a certain distance, then its neighboring vertices have to be moved a relatively similar distance - subject to the choice of the smoothing parameter Δi . For the first iteration, where we take s_0 steps, we set $\Delta i = 10$ to allow for more flexibility when expanding the mesh.

When the segmentation has been made, we essentially have a somewhat smooth estimation of how far each mesh vertex needs to be shifted along its normal in order to lie just on the inside of the skull. The mesh vertices are therefore shifted to the corresponding coordinate in $\mathbf{C}$, that lies on the transition from foreground to background in the segmentation, along each mesh vertex normal.

After the coordinates of the mesh vertices have been updated, 10 iterations of Taubin smoothing are applied to the mesh. This is rather aggressive, but maintaining a smooth surface is crucial for the robustness of our method, and the smoothing only affects intermediate results, not the final mesh representing the endocast. If the mesh is not kept smooth, the normal vectors of the mesh vertices could overlap, leading to unwanted results when performing the segmentation. Taubin smoothing is chosen, as it does not cause the same amount of mesh shrinkage that for example Laplacian smoothing does [27]. This is important as we're growing a mesh out from a point, and any backwards pulls are essentially delaying the algorithm.

The result of the first iteration of mesh expansion on a wolf μ CT-scan is seen in Figure 27 on the next page.

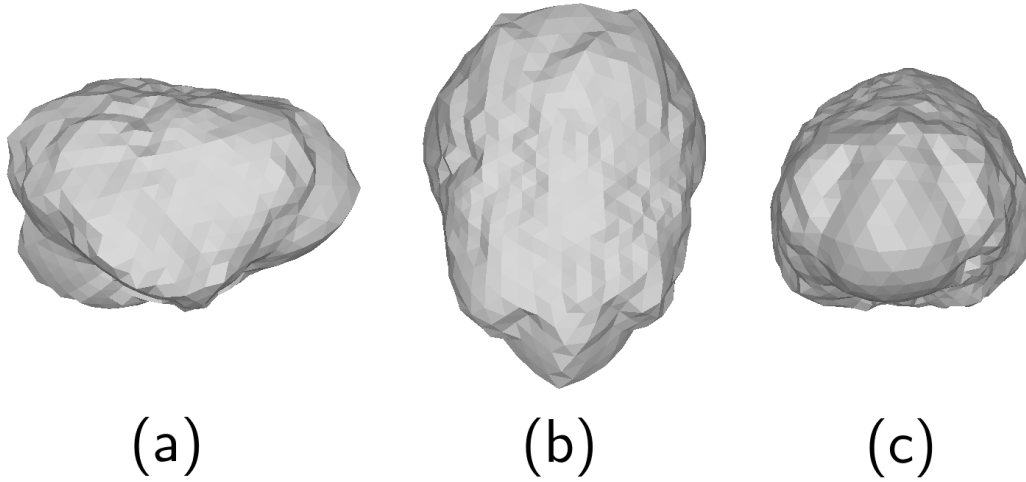


Figure 27: A resulting mesh after the first iteration on a 0.5 times downsampled wolf μ CT-scan as seen from the side (a), the top (b) and the front (c).

Before the next iteration begins, the mesh is subdivided using loop subdivision [28]. This results in four times as many vertices, and thus a larger amount of flexibility in the mesh when doing the next iteration.

From this new mesh, the procedure of sampling along the normal vectors of the mesh vertices, constructing a graph, finding a minimum cut, and then once again updating the mesh vertices and smoothing, is done iteratively with only a few changes, as will now be covered.

5.4.4 Difference between the first iteration and the following ones

The first iteration samples s_0 steps, as defined in (22), along the normal vector. After this iteration, the mesh will have adapted much better to the endocranium, as seen in Figure 27, meaning we do not have to sample as many steps in the coming iterations.

As the mesh is now quite close to the inside layer of the skull, we wish to avoid that the layer slowly drifts into the skull. Therefore, for all coming iterations we do not only sample along the normal vectors, but also in the opposite direction. This allows for the opportunity to 'redo' mistakes in the fitting to the inside of the skull. We take the same amount of steps in each direction. We have found that taking what corresponds to 10 mm's of steps, with a step size of 1 unit normal vector, in each direction works well in most cases for the first set of iterations, before the mesh is subdivided again. This is a trade off, as it limits the mesh's ability to expand quickly into areas not yet caught, but it also limits the mesh's ability to expand into possible holes in the skull - for example the *foramen magnum*.

Once the mesh is subdivided, a new step size is chosen. In our test cases, choosing 0.5 for the remaining iterations works well. This results in now only sampling 5 mm's in each direction when creating the column-image. As the mesh converges quite quickly, limiting the search space quickly works well.

Depending on the resolution of the μ CT-scan, the mesh should be subdivided a certain amount of times in the intermediate iterations. If the volume has a high value voxel spacing (i.e. each voxel corresponds to a larger amount of millimeters in the real world) the mesh does not have to be as high quality, as the amount of details captured in such a volume is not very high. On the other hand, if the volume has a low value voxel spacing, it will have captured very precise qualities of the skull, meaning the endocast can be made to capture these as well, if we subdivide the mesh enough times.

We found that applying 3 intermediate loop subdivisions for an isotropic NIfTI volume, meaning it has the same voxel spacing along all axes, with a voxel spacing of $\approx 0.24 \frac{\text{mm}}{\text{voxel}}$, resulted in a high quality mesh that accurately casted the endocranium of the skull. A volume with a voxel spacing of $\approx 0.48 \frac{\text{mm}}{\text{voxel}}$ needs only 2 loop subdivisions of the mesh through the iterations.

Pseudo code of the iterative mesh expansion can be seen in Algorithm 1. It takes in two inputs: `num_iters` and `step_sizes`, which are both lists. The elements in `num_iters` are the number of iterations that should be performed between each mesh subdivision. The elements in `step_sizes` are the step sizes (in unit normal vectors) corresponding to each set of iterations in `num_iters`.

For an isotropic volume with a voxel spacing of $\approx 0.24 \frac{\text{mm}}{\text{voxel}}$ we use:

- `num_iters` = [10, 5, 3, 2],
- `step_sizes` = [1, 1, 0.5, 0.5],

in our test cases. This means that the mesh is subdivided a total of 4 times; one initial after the first iteration and once between each element in `num_iters`.

Algorithm 1 Iterative Mesh Expansion

```

1: Input: step_sizes, num_iters
2: for  $i = 0$  to  $\text{length}(\text{num\_iters}) - 1$  do
3:   for  $k = 0$  to  $\text{num\_iters}[i] - 1$  do
4:     if  $i = 0$  then
5:        $\Delta i \leftarrow 4$ 
6:     else
7:        $\Delta i \leftarrow 1$ 
8:     end if
9:      $\text{step\_size} \leftarrow \text{step\_sizes}[i]$ 
10:    Generate column-image based on  $\text{step\_size}$  ▷ See Section 5.4.1
11:    Perform graph cut segmentation
12:    Shift mesh vertices ▷ See Section 5.4.3
13:    Apply Taubin smoothing
14:  end for
15:  if  $i < \text{length}(\text{num\_iters}) - 1$  then
16:    Perform loop subdivision
17:  end if
18: end for

```

After the final iteration, a mesh like the one seen in Figure 28 on the next page is obtained. It captures a detailed cast of the endocranium in the volume. However, the mesh does also expand out through the *foramen magnum* at the back creating a strange ball-shape.

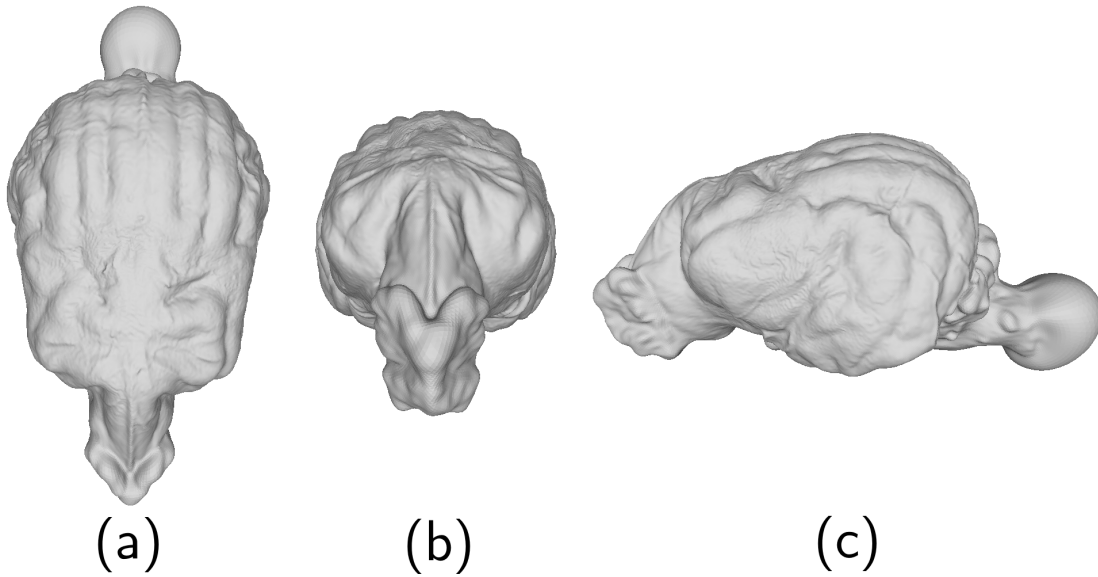


Figure 28: A resulting mesh after the last iteration on a 0.5 times downsampled wolf μ CT-scan seen from the top (a), the front (b), and the side (c).

With this mesh, a new binary volume - with the same dimensions as the original one from which the endocast was extracted - can be created. In the same way as a binary mask was created for the points inside of the convex hull mesh earlier, a binary mask is created here of the points in the volume that lie within the found endocast mesh. This can be used as a direct segmentation of the endocranium, and will also be used for method validation in the results section.

The code for our implementation can be found on GitHub at reference [29].

6 Results

In this section we will go over the settings we used to extract the endocasts, that we found to be detailed and precise. We will also showcase different final endocast examples as well as highlight certain aspects of them that are a result of our method.

As mentioned in section 5.4.4 we found that using the settings:

$$\begin{aligned}\text{num_iters} &= [10, 5, 3, 2], \\ \text{step_sizes} &= [1, 1, 0.5, 0.5],\end{aligned}$$

for an isotropic volume with a voxel spacing of $\approx 0.24 \frac{\text{mm}}{\text{voxel}}$ worked well. Δi is set to 10 in the initial icosphere expansion and then changes as shown in Algorithm 1 earlier. All of these results were created on the DTU high performance computer on which it took about 600 seconds to do PCA-resampling of the volume and finding candidate points, as well as roughly 1000 seconds to do the iterative mesh expansion. Figure 29 shows an example of how a mesh expands with these settings.

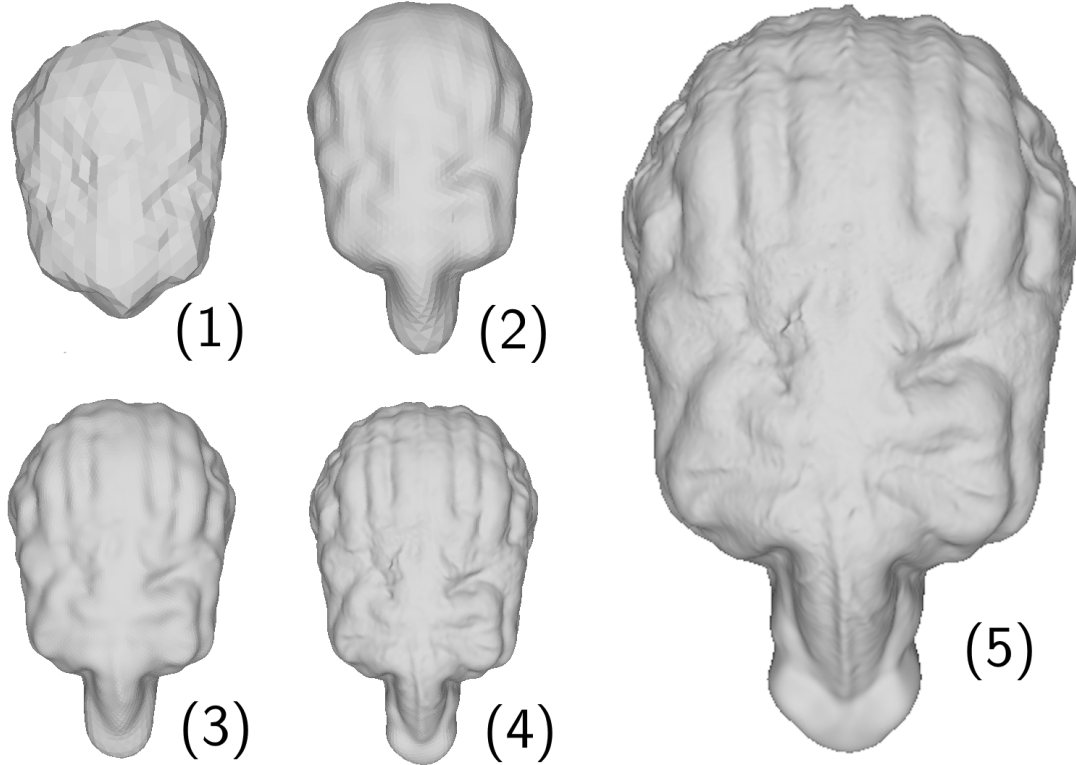


Figure 29: A showcase of the way the mesh expands, going from start to finish. (1) Initial icosphere expansion of the mesh. (2) The last iteration of the first subdivision. (3) The last iteration of the second subdivision. (4) The last iteration of the third subdivision. (5) Final result after the fourth subdivision.

6.1 Results analyzed

To verify the accuracy of the extracted endocrania, we visually compared them against the original μ CT-scans by overlaying the resulting binary segmentation and assessing their anatomical correspondence. We did this for 4 different of the supplied wolf species μ CT-scans. These results can be seen in the following Figures 30, 31, 32, and 33.

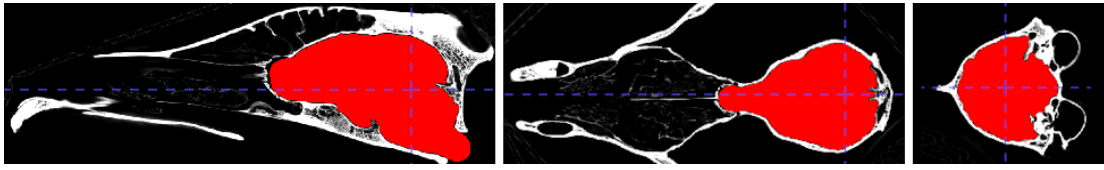


Figure 30: The extracted endocranium of BM_2600 can be seen overlaid in their original μ CT-scan.

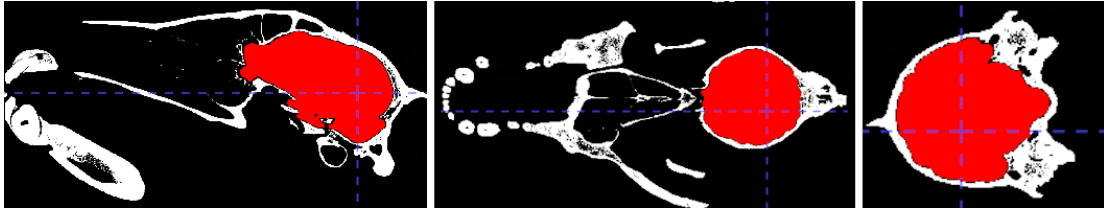


Figure 31: The extracted endocranium of NRM_20105021 can be seen overlaid in their original μ CT-scan.

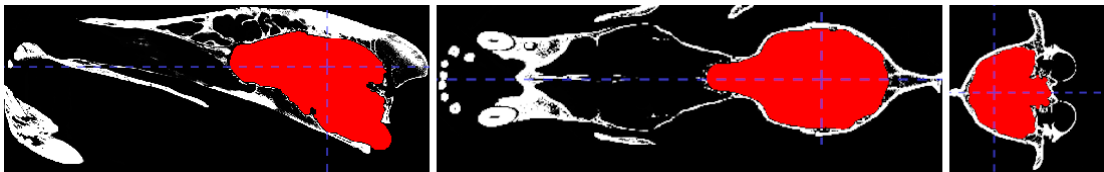


Figure 32: The extracted endocranium of NRM_A586548 can be seen overlaid in their original μ CT-scan.

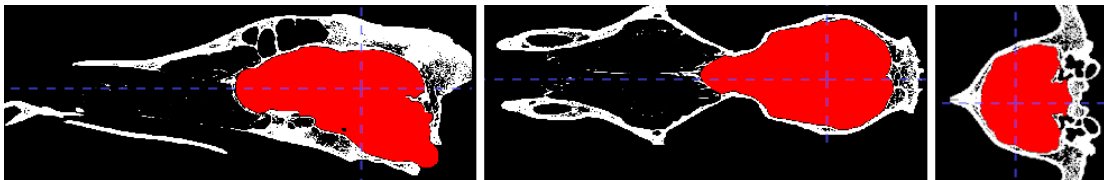


Figure 33: The extracted endocranium of UAM_47189 can be seen overlaid in their original μ CT-scan.

In these four figures we generally see that the extracted endocast aligns closely with the original μ CT-scan. There are a few key details to highlight, namely:

1. Endocasts don't always fully extend to the inner surface of the neurocranium.
2. The mesh will continue to extend out through holes in the neurocranium.

Examples of these can be seen in Figure 34 on the next page.

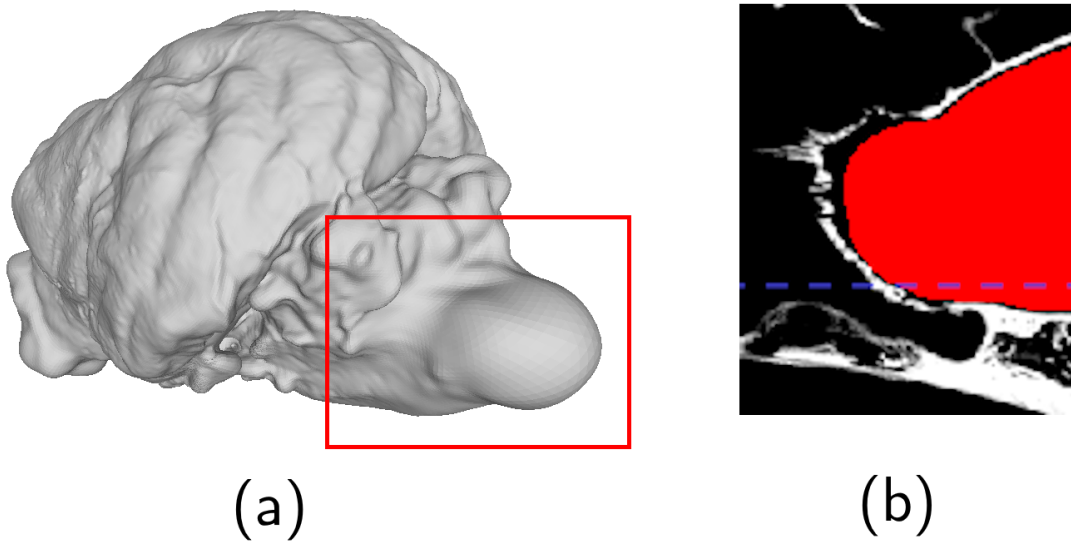


Figure 34: (a) An artifact that is left as a result of the mesh growing through the *foramen magnum*. (b) An illustration of the mesh not expanding all the way into crevices.

A collection of the endocasts of the four wolves analyzed in this project can be seen in Figure 35.

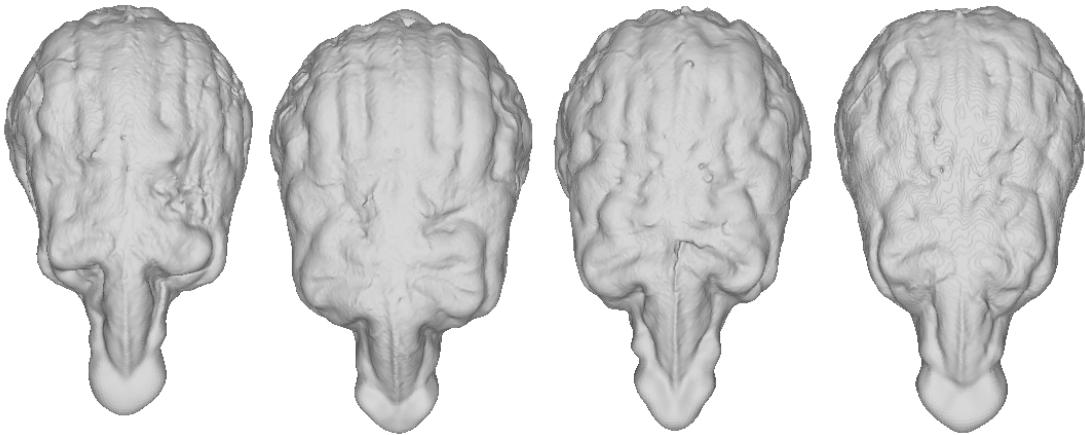


Figure 35: A showcase of the endocasts produced by our method on 1000^3 voxel volumes of, from left to right, BM_2600, NRM_20105021, NRM_A586548 and UAM_47189.

6.2 Runtimes and mesh resolution

Table 1 shows the runtime of our proposed method in different cases: on a 500^3 voxel volume (meaning lower resolution) of NRM_20105021, on a 1000^3 voxel volume of NRM_A586548, and on the same 1000^3 voxel volume with an extra final subdivision of the mesh and one extra iteration. The exact settings for each of these can be seen in appendix B.1.

Resolution	PCA-resample	Find cand. points	Mesh expansion	Total
Wolf_500	52.43	22.25	164.32	239
Wolf_1000	419.80	201.32	1050.11	1671.23
Wolf_1000_V2	419.80	201.32	2453.34	3074.46

Table 1: This table shows how long each process takes in seconds for each of three cases. 1) A volume of the wolf NRM_20105021 with 500^3 voxels (WOLF_500). 2) A volume of the wolf NRM_A586548 with 1000^3 voxels (WOLF_1000) and 3) A of the wolf NRM_A586548 with 1000^3 voxels with one additional subdivision in the endocast creation (WOLF_1000-V2).

We also tried testing on the full-resolution 2000^3 voxel volume. However, even with 300 GB of available memory, it was not enough to be able to run our method.

Almost twice the amount of time was needed to get an extra iteration with one more subdivision. A side-by-side of the differences of each of the three endocasts can be seen in Figure 36. From this, we see that there is a pretty significant gain in the level of detail when going from the lower resolution volume to the higher one. We also note that the extra subdivision and iteration does improve upon the mesh, but not to a nearly as high degree as going from low resolution volume to high resolution volume.

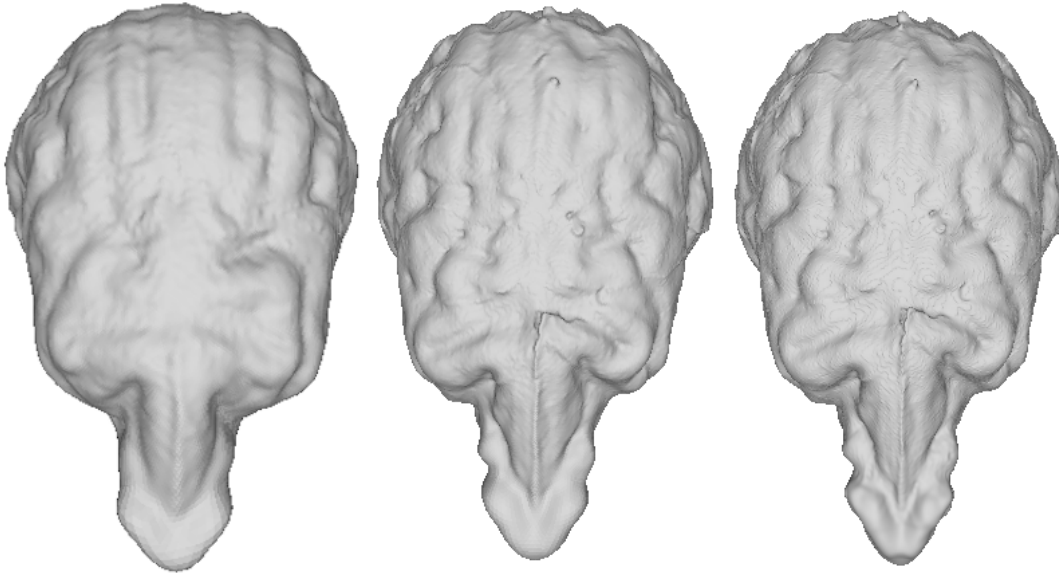


Figure 36: A top-down view of each of the three different cases with the endocast on the left side being from NRM_20105021 downsampled to 500^3 voxels (Wolf_500), the middle being NRM_A586548 downsampled to 1000^3 voxels (Wolf_1000), and the right being NRM_A586548 downsampled to 1000^3 voxels with an extra subdivision and iteration in the endocast creation (Wolf_1000_V2).

The part of the endocast that is the most different between the three aforementioned meshes is the front part, as seen in Figure 37. It is clearly seen that this part is captured in a much higher level of detail when the extra subdivision and iteration is performed. Why this is will be touched upon in more detail in the discussion.

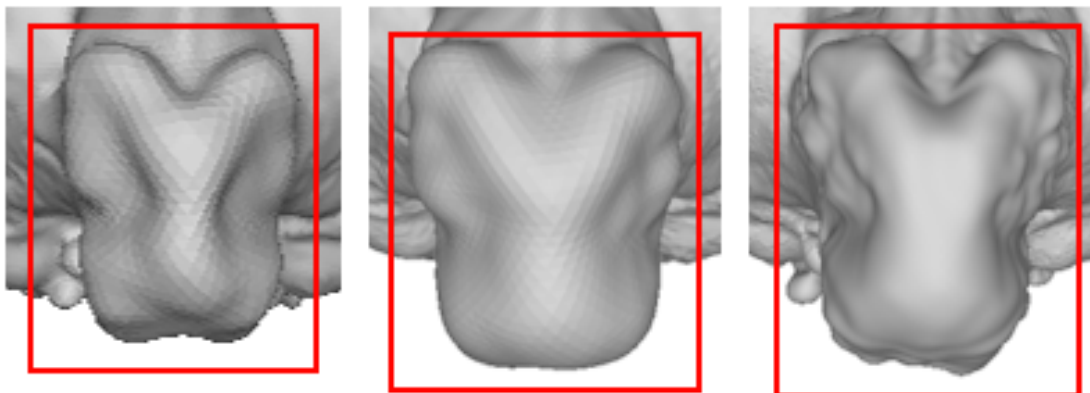


Figure 37: Comparison of different qualities of meshes at the olfactory bulbs. The endocasts correspond directly to those of Figure 36.

7 Discussion

7.1 Did we solve the problem?

7.1.1 Results

In section 6 we saw that our proposed method for digital endocast creation is capable of producing visually plausible endocasts from μ CT-scans of wolf skulls. The produced endocast meshes are able to capture even fine details, such as what appears to be blood vessels running on top of the brain. As Figure 36 shows, the level of detail captured in the produced mesh endocast is dependent on the level of detail in the input volume, meaning its voxel resolution. Thus, the level of detail in the final endocast is directly dependent on the resolution of the initial μ CT-scan.

We have seen that our proposed method is capable of extracting endocasts from volumes of dimensions $500 \times 500 \times 500$ in a matter of minutes. We have also seen that it is capable of extracting endocasts from volumes of dimensions $1000 \times 1000 \times 1000$, even locally on a laptop with 32 GB of RAM, though the amount of time spent increases rapidly, taking just under 30 minutes for the full procedure on a $1000 \times 1000 \times 1000$ volume when executed on a High Performance Computing cluster, though it was run on a node with shared resources. Running on $2000 \times 2000 \times 2000$ volumes was not possible in our testing, as even 300 GB of RAM was not enough to accomplish this. Overall, this does show great promise for the method, as one can extract a relatively good representation of an endocast from a μ CT-scan on a local machine within minutes. And, if time is not of the essence, a more detailed version, given 32 GB of ram is available. We saw in Figure 36 and in Table 1 that doing an extra subdivision on a volume with voxel resolution $\approx 0.24 \frac{\text{mm}}{\text{voxel}}$ did not add a lot of extra details - except at the olfactory bulbs - but increased the run time by a factor of almost 2, making it likely not worth it.

Our proposed method should not require any manual tweaking if the volume supplied has a voxel resolution of $\approx 0.24 \frac{\text{mm}}{\text{voxel}}$, which can easily be achieved by downsampling the volume if it has a voxel resolution of less than $0.24 \frac{\text{mm}}{\text{voxel}}$. However, if the voxel resolution deviates significantly from this, other parameters, specifically Δi should be tweaked to reflect this.

7.1.2 Strengths of our method

Detailed, watertight meshes: A strength of our proposed method is its ability to create watertight meshes accurately capturing details of the endocranium within reasonable time. It produces great visual results and can be adapted to higher or lower resolutions when appropriate, though it takes some manual tweaking.

User-friendly graphical user interfaces: Another strength of our method is its close to automated nature. In its current version, the user has to interact with two simple graphical user interfaces, making it still valid for user not as familiar with the more technical aspects of the method. The two steps that are not automated can likely be automated, but they are outside of the scope of this project. This will be touched upon in more detail later in the discussion.

Strong proof of concept: Our method, in its current form, stands as a strong proof of concept for this approach to digital endocast creation from μ CT-scans of animal skulls. It is already capable of producing strong visual results, and many of its current shortcomings can likely be resolved by further work, as will be addressed later in the discussion.

7.1.3 Shortcomings of our method

Memory usage: A main shortcoming of our proposed method in its current form is its vast memory usage when running. It is unable to run on volumes with a large number of voxels, unless huge amounts of RAM is available. In our case this has meant that we have not been able to extract a full-resolution endocast from the full-resolution μ CT-scans, of voxel spacing

0.12 $\frac{\text{mm}}{\text{voxel}}$, we had available. This would likely have captured way more fine details due to the much higher resolution.

Dependency on human interaction: Though our method currently resolves the issue of not being fully automatic by implementing graphical user interfaces, it would still have been ideal to have it fully automatic such that the user only needs to supply a μ CT-scan and the method returns the endocast. This would allow for processing of large amounts of data way more frictionless, as it would, in its current version, require a human to interact with it and make choices for all μ CT-scans supplied.

Largest connected component assumption: Our method currently assumes that the neurocranium will be a part of the largest connected component in the binary volume after the Otsu thresholding as described in section 5.1. This assumption might hinder the method from generalizing well, as there are likely animal skulls with widely different anatomical properties in which this assumption does not hold. Other approaches, such as utilizing binary morphology or simply dynamically deciding a BLOB-size threshold, could likely be used to avoid or reduce noise without making the same assumptions.

Heuristic for sampling radius: The heuristic we have used in our method for deciding the initial sampling radius from the icosphere, as described in section 5.4.1, also builds on a certain assumption about how much of the volume the endocranium takes up along each anatomical axis. This assumption is also not likely to hold true for all animal skulls, meaning there could be instances in which our method does not work as intended.

Voxel resolution sensitivity: The method we propose is quite sensitive to the voxel resolution of the input volume, as especially Δi needs to be tweaked correctly to allow for the correct amount of flexibility of the mesh when expanding. As Δi is not chosen dynamically in our method in its current form, the method can easily fail to produce valuable results if the voxel resolution is not what is expected.

Hard-coded parameters: This leads to the next point, which is that a lot of parameters are chosen more or less to fit wolves specifically in the current version of our method. This means that the method does not generalize to work for all skulls in the current version. It lacks automated ways of picking these parameters so that they fit each specific case. This could be, for example, choosing Δi , the step size, or the number of iterations between each subdivision.

Voxel intensity sensitivity: We have also found, that our method can be sensitive to the voxel intensity distribution in the original μ CT-scan input. Sometimes, there are a very small set of voxels with relatively much higher intensities than the rest which can have a great impact on the result of the Otsu thresholding and graph cut segmentation. This happens, as the graph that is set up will not work as intended, as it bases its values on the highest voxel intensity, which will not at all be representative in this case.

Possibility of vertex normal overlap: One last thing to touch upon is the potential for vertex normal overlap. Currently the mesh is relatively aggressively smoothed in every intermediate step. But, this does not guarantee that vertex normals will not overlap eventually if they are sampled far enough out. If this happens, the mesh is at risk of folding on itself, leaving a strange artifact on the final mesh - or potentially completely ruining it. A method that could be implemented to fix this in future work is mentioned later in the discussion.

7.1.4 Comparison to Endomaker and SlicerMorph

As mentioned in section 3.3, we found certain shortcomings in the field of digital endocast creation. Generally, we found the current existing methods we tested, that being Endomaker and SlicerMorph, quite difficult to get working correctly and producing good visual results.

For SlicerMorph we found that we lacked a better documentation of what exactly their method did. This also lead to a very difficult time when trying to tune the parameters of the method,

as we were unsure of what exactly each parameter influenced and should be chosen to be - even though there were only 2 of them.

For Endomaker we found it to be more well documented and well described, meaning we got a better understand of how it worked and what we could do to get better results. However, we struggled with a lot of error codes when trying to use it to create endocasts from our μ CT-scans. Between attempts the resulting output would change. Sometimes it would create a cast of the jaw bone and sometimes we would get the endocast. For all cases, though, the resulting mesh was definitely not watertight nor visually completely satisfying.

Overall we found it difficult to get the current methods to consistently produce good results if even at all produce good results. Limited amounts of information about how different parameters should be tweaked prevented us from helping mitigate this ourselves. We also found that the methods seemed quite sensitive to the input, putting more work on the shoulders of the user to provide correct input, for example cropped volumes.

Our proposed method produces more visually appealing and detailed endocasts in about the same run time. It is able to capture the full endocranium, given enough iterations and sampling distance. In its current form, the user does not need to tweak any parameters, as long as the input matches the desired voxel spacing. They just have to perform simple interactions with graphical user interfaces. This should make it much easier for someone with limited experience or knowledge to get going when using our method as opposed to the already existing ones. We find that our method has shown great potential, and there are many aspects that can likely be improved upon in the future, making it generalize even better, run at faster speeds, and on even larger volumes. We also discuss some of these later on. Our method does, however, still have its shortcomings. There is definitely still things to be done in this field, but we have proposed a new direction in which to search for a more stable method for digital endocast creation.

7.2 Alternative methods and future work

7.2.1 Method refinements

Even mesh vertex density: As was illustrated in Figure 37, the current version of our proposed method does not have an ideal way of handling when a non-negligible part of the endocranium is not directly visible along the rays of the vertex normals for the initial icosphere or there are deep crevices that need to be filled. The nature of our method results in only a certain amount of faces being 'allocated' to eventually fill out the volume that was not directly visible initially or the part of the volume that is in a deep crevice. An illustration of this can be seen in Figure 38. This means that more subdivisions of the mesh will have to be made to eventually make sure it is fine enough in this region to accurately fill out and represent the endocranium in details. However, this is both computationally expensive, as the entire mesh is subdivided, meaning we are paying a huge global computational price to only really improve a small local part. Another result of this is that the mesh can simply become too fine in the other parts of the endocranium, leading to some relatively odd behavior.

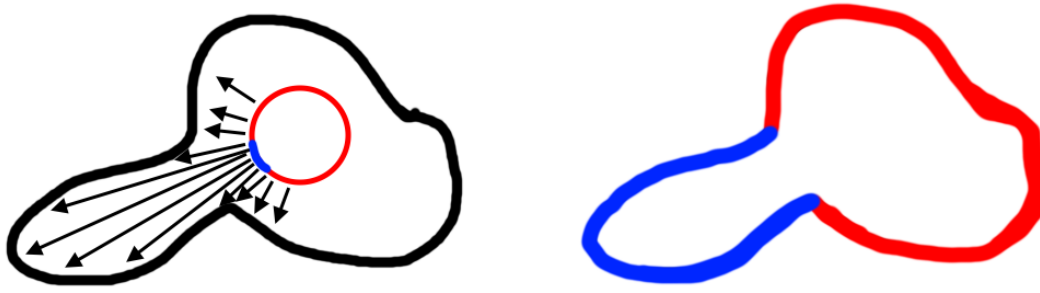


Figure 38: An illustration of how a relatively small amount of the mesh faces can end up having to make up a much larger part of the final endocast.

To alleviate this problem, multiple approaches can be tried in future work. One proposal would be to perform remeshing, perhaps uniform remeshing, at the intermediate steps to better even out imbalances in vertex density throughout the mesh [30]. Another approach would be to apply subdivision only to certain faces that have a large area throughout the process.

More optimized initial sampling distance: At the moment the initial sampling distance along the vertex normals of the icosphere is based on a heuristic which uses information about the size of the entire volume to decide a safe sampling distance. We do, however, already have a segmentation of the skull, meaning we could reduce the search space to only analyze the dimensions of the skull rather than that of the entire volume, potentially making it more efficient.

Scaling to larger volumes: We are quite confident in the feasibility of making our method capable of analyzing even a $2000 \times 2000 \times 2000$ μ CT-scan. This could be done by initially performing the endocasting in a downsampled version of the volume to obtain an estimate of the endocast. This could then be scaled up to the corresponding area in the full-resolution volume. From this, the search space could be greatly reduced, as the endocast would already be close to done, meaning we need only the sub-volume that is bounding box of the current estimate in the full-resolution volume - perhaps with some extra padding as a precaution. In the case of wolf skulls the endocranium does not actually take up a large amount of the total volume, meaning in the $2000 \times 2000 \times 2000$ volume, the sub-volume containing the endocranium would likely not even be of size $1000 \times 1000 \times 1000$, meaning our method should easily be able to run on this, if it could be extracted. This does, however, require great book-keeping of the different transformations happening throughout the process. This would be of great interest for future work on our method.

Robust adaptive choice of Δi : The choice of Δi for the graph construction heavily influences the obtained result. The voxel resolution of the volume one is working with, as well as the current step size when sampling along the mesh vertex normals, both directly influence what the choice of Δi means in terms of smoothness in world space. Therefore, Δi should be chosen dynamically based on these factors. The exact way in which it should be chosen as to yield robust results is unclear at the moment. Perhaps it should be chosen to correspond to some real world mm distance. However, this would likely only work for very vertex-dense meshes, as there would otherwise not be a guarantee that the same step along two neighboring vertex normals would actually lie close to each other in real world space. Anyhow, future work in this would greatly add to the robustness and generalization of our proposed method.

Voxel intensity variance sensitivity: As mentioned earlier in the discussion, our method in its current form can be sensitive to outlier voxel intensities in the μ CT-scan volume. One way of possibly handling this would be to run a Z-test on the voxel intensities to search for outliers [31].

These found outlier values could then be scaled to the lowest or highest voxel intensity that is not deemed to be an outlier depending on which side of the distribution the voxel intensity lies.

Locating only one endocranium center point: When searching for the center point of the endocranium, our method looks for local maxima in the distance transformed volume of the skull. The implementation it is currently using has a parameter deciding the neighborhood size, as described in section 5.3.2. This should be decided in a dynamic way dependent on the input volume, as making it as large as possible greatly reduces the number of found candidate center points. Perhaps, it would even be possible to make it locate only the one true endocranium center point, making the user have to make one interaction less. More work looking into this would help achieve the goal of not needing human interaction.

7.2.2 Alternative approaches

Avoiding NIfTI requirement: Part of our current pipeline is converting the supplied `.vol` and `.vgi` files to the NIfTI format. This is, strictly speaking, not needed, as it does not add any new information about the volumes. It is simply another representation. Therefore, one could likely easily take another approach to our method in which the input is simply those initial files - or perhaps it would be able to accept multiple types. This would make the method more available to more people as the user would not have to know about, or worry about, different file formats for μ CT-scan volumetric data.

Avoiding PCA-resampling: Another step in our pipeline, that is not a strict necessity, is the PCA-resampling of the volume to align the standard anatomical axes with the axes of the volume. This quickly becomes quite computationally heavy for larger volumes, making it infeasible. The PCA resampling is not needed in order to locate the endocranium center point nor is it needed for expanding the mesh from that point. It only plays a role in aligning the volume to be easier to interpret for the user and for heuristics such as choosing the initial sampling range along the mesh vertex normals. This could, however, likely easily be chosen in another way such as using information about the skull point cloud bounding box. Avoiding the PCA-resampling would also have the benefit of removing one of the human interactions needed to create the endocast, pushing it towards becoming more automatic.

Preventing growth through holes in skull: Our current method does not handle holes in the endocranium, such as the *foramen magnum*, very well. The mesh will slowly expand out through the hole for each iteration taken. This leads to incorrect artifacts such as the ones seen in Figure 34a. This is not a huge problem, as the resulting meshes are primarily meant for humans to view and draw conclusions from, and a human would be able to ignore these artifacts. If the goal would at some point be to be able to feed them into, for instance, a deep learning model, this problem in our method would need to be addressed. This could potentially be done by adding intra-column edges between neighbors in the same layer in the graph that is constructed, to make it favor staying at the current level if it does not hit a layer of skull. This would be a drastic change in the way the graph cut segmentation is done though, so much further work and analysis would need to be conducted.

Using alpha shapes instead of convex hull: Currently, a convex hull of the skull point cloud is used to restrict the region of interest when trying to locate the center point of the endocranium. Another, and perhaps better approach, would be using what are called alpha shapes. Alpha shapes are a generalization of the convex hull of a point set. It is a Delaunay triangulation of the point cloud which discards any triangles whose circumsphere radius is larger than a supplied α threshold. The remaining triangles form the alpha complex. The surface triangular faces of this alpha complex make up the alpha shape, which captures the concavities of the point cloud at a scale controlled by α [25]. Using this method, rather than just the convex hull, could potentially limit the ROI even more. However, in our testing, the implementations that produced the meshes of these shapes often had very irregular face normals directions, leading to problems when testing which points were inside or outside of the shape. We do believe

though, that it would be possible to make this work to create an even tighter ROI in which the endocranium center point would lie. This would, again, help reach a point in which the localization of the center point would be completely automatic.

Avoiding normals overlapping: One last different approach, would be using Gradient Vector Flow to avoid the risk of mesh vertex normals overlapping, resulting in mesh folding. The gradient vector flow field can be used to construct non-overlapping search profiles. Implementing this, rather than simply sampling along the normal vectors of the mesh vertices, the sampling would be done along a vector field of the gradients, meaning no overlaps would ever be possible. This method has shown great results in other optimal surface segmentation tasks [32].

7.2.3 Running locally

It would be great if our method could be run locally on a laptop or desktop rather than only being able to run on a HPC - even for larger volumes. In order for this to be possible, memory-handling would need to be improved in the code. This has not been a priority for this project, but it would be worth looking into for further work on our proposed method.

The same goes for the code in general. Writing efficient and optimal code has been outside the scope of this project, meaning the code that has been used and tested in the results section can likely be improved upon through vectorization of computations, avoiding redundant computations through better passing of already computed values and variables. There is a good chance that the method could see a non-negligible increase in speed and memory efficiency if it were to be looked into with more time. Also, improved garbage collection in the code, meaning freeing memory that is no longer needed, could greatly benefit the code - especially as memory is already a limiting resource for our method.

7.3 Perspective

We have laid the groundwork for a new approach to creating digital endocasts from μ CT-scans of primarily wolf skulls. With further work our method can likely be fit to work on a larger variety of animal skulls more robustly. Our method can be provide biologists with accurate and detail visual representations of endocasts from μ CT-scans, helping them in their research on comparative anatomy, evolutionary biology or paleoneurobiology. It can help in drawing conclusions about brain size, shape and volume estimations, as well as for studying differences in relative size of regions in the brain between species. Examinations by biologists of the results produced by our method can possibly lead to conclusions about evolution in different species.

8 Conclusion

In conclusion, we found the existing methods for digital endocast creation that we evaluated to be somewhat fragile and lacking in user guidance - challenges that may be even more noticeable for users who do not have software-technical or image and mesh processing backgrounds. Therefore, we proposed a new novel approach to digital endocast creation from μ CT-scans of skulls, with the aim of tackling some of these current challenges. Our mesh-based approach utilizes a found center-point of the endocranium from which it expands a certain distance outwards found by a graph cut segmentation.

In our testing, we found that our method was capable of yielding visually plausible and pleasing endocasts from μ CT-scans of wolf skulls, as long as the number of voxels in the volume did not significantly exceed 1000^3 . When this was the case, our method was able to run - even locally on laptops, as long as 32 GB of RAM was available. The amount of detail that is possible to capture in the resulting endocast depends heavily on the voxel resolution of the input volume. The amount of detail captured depends on the number of iterations and subdivisions applied by our method. We found that our method, though not yet fully optimized for memory usage and runtime, was still able to perform within the same time frame as the existing methods in our test cases.

The focus of this thesis has been on wolf endocasting. Therefore, our method in its current form is tailored to this. Certain parameters would, as discussed, likely need to be changed to work equally well on other species. These include the smoothing parameter Δi and the number of iterations and subdivisions applied. Currently, human interaction is needed at two points throughout the pipeline, which is not ideal. They have, however, been implemented to be very user-friendly to accommodate users of different technical backgrounds, though they have not been tested on user in practice.

A notable shortcoming of our proposed method is its difficulty in capturing crevices within the endocranium that have a narrow entrance and open up into larger internal cavities. This limitation stems from how our method assigns mesh faces toward the opening, resulting in a low resolution, as was often the case for the olfactory bulbs in the wolf endocast.

While our discussion highlights more shortcomings than strengths, this primarily reflects the many ideas we have for improving the method — rather than suggesting it is fundamentally lacking. We are pleased with the results achieved and view them as a strong proof of concept. With further research and development, such as the directions we have proposed, we believe the method holds significant potential for improvement and great promise for contributing with value to the field of digital endocasting.

Our thesis leaves room for further research into the development of our proposed method. We have proposed possible improvements that could be looked into and implemented in future work, making the method more robust and general, allowing it to be used in a much broader variety of cases.

References

- [1] Elsevier. *Endocast – an overview*. <https://www.sciencedirect.com/topics/veterinary-science-and-veterinary-medicine/endocast>. ScienceDirect Topics (Elsevier). Accessed: 03-06-2025.
- [2] Emily A. Buchholtz and Ernst-August Seyfarth. “The Study of “Fossil Brains”: Tilly Edinger (1897–1967) and the Beginnings of Paleoneurology”. eng. In: *Bioscience* 51.8 (2001), p. 674. ISSN: 00063568, 15253244.
- [3] T. Edinger. *Die fossilen Gehirne*. Ergebnisse der Anatomie und Entwicklungsgeschichte. J. Springer, 1929. URL: <https://books.google.dk/books?id=R5KSGQAACAAJ>.
- [4] Pascaline Lauters, Martine Vercauteren, and Pascal Godefroit. “Chapter 1 - Endocasts of ornithomimid dinosaurs: Comparative anatomy”. In: *From Fossils to Mind*. Ed. by Tanya Calvey, Alexandra A. de Sousa, and Amélie Beaudet. Vol. 275. Progress in Brain Research. Elsevier, 2023, pp. 1–23. DOI: <https://doi.org/10.1016/bs.pbr.2022.10.002>. URL: <https://www.sciencedirect.com/science/article/pii/S007961232200187X>.
- [5] Amy M. Balanoff et al. “Best practices for digitally constructing endocranial casts: examples from birds and their dinosaurian relatives”. eng. In: *Journal of Anatomy* 229.2 (2016), pp. 173–190. ISSN: 14697580, 00218782. DOI: [10.1111/joa.12378](https://doi.org/10.1111/joa.12378).
- [6] A. Profico et al. “endomaker, a new algorithm for fully automatic extraction of cranial endocasts and the calculation of their volumes”. In: *American Journal of Physical Anthropology* 172 (3 2020), pp. 511–515. DOI: [10.1002/ajpa.24043](https://doi.org/10.1002/ajpa.24043).
- [7] Sarah Rolfe et al. “SlicerMorph: An open and extensible platform to retrieve, visualize and analyse 3D morphology”. In: *Methods in Ecology and Evolution* 12.10 (2021), pp. 1816–1825. DOI: [10.1111/2041-210X.13669](https://doi.org/10.1111/2041-210X.13669).
- [8] Kang Li et al. “Optimal Surface Segmentation in Volumetric Images-A Graph-Theoretic Approach”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* (2006). DOI: [10.1109/TPAMI.2006.19](https://doi.org/10.1109/TPAMI.2006.19).
- [9] Sharleen T Sakai et al. “Endocranial Development in the Coyote (*Canis latrans*) and Gray Wolf (*Canis lupus*): A Computed Tomographic Study”. In: *Brain, Behavior and Evolution* 91 (Apr. 2018). DOI: [10.1159/000487427](https://doi.org/10.1159/000487427).
- [10] Vedrana Andersen Dahl, Anders Bjorholm Dahl, and Rasmus Larsen. “Surface Detection using Round Cut”. eng. In: *Proceedings of the International Conference on 3d Vision (3dv 2014)* 2 (2014), pp. 82–89. DOI: [10.1109/3DV.2014.60](https://doi.org/10.1109/3DV.2014.60).
- [11] Micro Photonics Inc. *What is Micro-CT? An Introduction*. <https://www.microphotonics.com/what-is-micro-ct-an-introduction/>. Accessed: 20-05-2025. 2020.
- [12] BlueQuartz Software, LLC. *Read Volume Graphics File (.vgi/.vol) Filter*. Version 7.2.0. Dream3D-NX. 2024. URL: https://www.dream3d.io/nx_reference_manual/html/SimplnxCore/ReadVolumeGraphicsFileFilter.html.
- [13] Bane Sullivan and Alexander Kaszynski. “PyVista: 3D plotting and mesh analysis through a streamlined interface for the Visualization Toolkit (VTK)”. In: *Journal of Open Source Software* 4.37 (May 2019), p. 1450. DOI: [10.21105/joss.01450](https://doi.org/10.21105/joss.01450). URL: <https://doi.org/10.21105/joss.01450>.
- [14] A. M. Winkler. *The NIFTI file format*. Accessed: 07-06-2025. 2012. URL: <https://brainder.org/2012/09/23/the-nifti-file-format/>.
- [15] Paul A. Yushkevich, Yang Gao, and Guido Gerig. “ITK-SNAP: An interactive tool for semi-automatic segmentation of multi-modality biomedical images”. eng. In: *2016 38th Annual International Conference of the IEEE Engineering in Medicine and Biology Society (EMBC)* 2016 (2016), pp. 3342–3345. ISSN: 26940604, 1557170x. DOI: [10.1109/EMBC.2016.7591443](https://doi.org/10.1109/EMBC.2016.7591443).
- [16] A. Profico et al. “Reproducing the internal and external anatomy of fossil bones: two new automatic digital tools”. In: *American Journal of Physical Anthropology* 166 (4 2018), pp. 979–986. DOI: [10.1002/ajpa.23493](https://doi.org/10.1002/ajpa.23493).

- [17] Stefan Van der Walt et al. “scikit-image: image processing in Python”. In: *PeerJ* 2 (2014), e453.
- [18] Andriy Fedorov et al. “3D Slicer as an Image Computing Platform for the Quantitative Imaging Network”. In: *Magnetic Resonance Imaging* 30.9 (2012), pp. 1323–1341. DOI: [10.1016/j.mri.2012.05.001](https://doi.org/10.1016/j.mri.2012.05.001).
- [19] SlicerMorph Team. *Endocast Creation Tutorial*. https://slicermorph.github.io/Endocast_creation.html. Accessed: 07-02-2025. 2021.
- [20] Henry R. Kang. *Computational Color Technology*. SPIE Press Monograph, 2006, pp. 155–157.
- [21] Nobuyuki Otsu. “A Threshold Selection Method from Gray-Level Histograms”. In: *IEEE Transactions on Systems, Man, and Cybernetics* 9.1 (1979), pp. 62–66. DOI: [10.1109/TSMC.1979.4310076](https://doi.org/10.1109/TSMC.1979.4310076).
- [22] Pauli Virtanen et al. “SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python”. In: *Nature Methods* 17 (2020), pp. 261–272. DOI: [10.1038/s41592-019-0686-2](https://doi.org/10.1038/s41592-019-0686-2).
- [23] Anatomy.app. *Neurocranium*. Accessed: 01-06-2025. 2025. URL: <https://anatomy.app/encyclopedia/neurocranium>.
- [24] Tue Herlau, Mikkel N. Schmidt, and Morten Mørup. “Principal Component Analysis”. In: *Introduction to Machine Learning and Data Mining*. Technical University of Denmark, Apr. 2023, pp. 33–48.
- [25] Herbert Edelsbrunner and Ernst Mücke. “Three-dimensional alpha shapes”. und. In: (1994), p. 32.
- [26] J. Andreas Bærentzen. *pygel3d*. Accessed: 01-04-2025. 2025. URL: <https://www2.compute.dtu.dk/projects/GEL/PyGEL/pygel3d.html>.
- [27] IEEE and G TAUBIN. “Curve and surface smoothing without shrinkage”. eng. In: *Fifth International Conference on Computer Vision, Proceedings* (1995), pp. 852–857. DOI: [10.1109/ICCV.1995.466848](https://doi.org/10.1109/ICCV.1995.466848).
- [28] Charles Loop. “Smooth Subdivision Surfaces Based on Triangles”. Accessed: 01-06-2025. Masters of Science. University of Utah, 1987. URL: <https://www.microsoft.com/en-us/research/wp-content/uploads/2016/02/thesis-10.pdf>.
- [29] Christian Bjerregaard and Aske Rove. *Endocasting GitHub repository*. Accessed: 10-06-2025. URL: <https://github.com/s224389/Endocasting>.
- [30] Sébastien Valette, Jean Marc Chassery, and Rémy Prost. “Generic remeshing of 3D triangular meshes with metric-dependent discrete voronoi diagrams”. eng. In: *Ieee Transactions on Visualization and Computer Graphics* 14.2 (2008), pp. 369–381. ISSN: 19410506, 10772626. DOI: [10.1109/TVCG.2007.70430](https://doi.org/10.1109/TVCG.2007.70430).
- [31] GeeksForGeeks. *Z-test : Formula, Types, Examples*. Accessed: 03-06-2025. 2025. URL: <https://www.geeksforgeeks.org/z-test/>.
- [32] Christian Bauer, Shanhui Sun, and Reinhard Beichel. “Avoiding mesh folding in 3D optimal surface segmentation”. eng. In: *Lecture Notes in Computer Science (including Sub-series Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)* 6938.1 (2011), pp. 214–223. ISSN: 16113349, 03029743. DOI: [10.1007/978-3-642-24028-7_20](https://doi.org/10.1007/978-3-642-24028-7_20).

9 Appendix

A SlicerMorph Pictures

A.1 SlicerMorph result 1

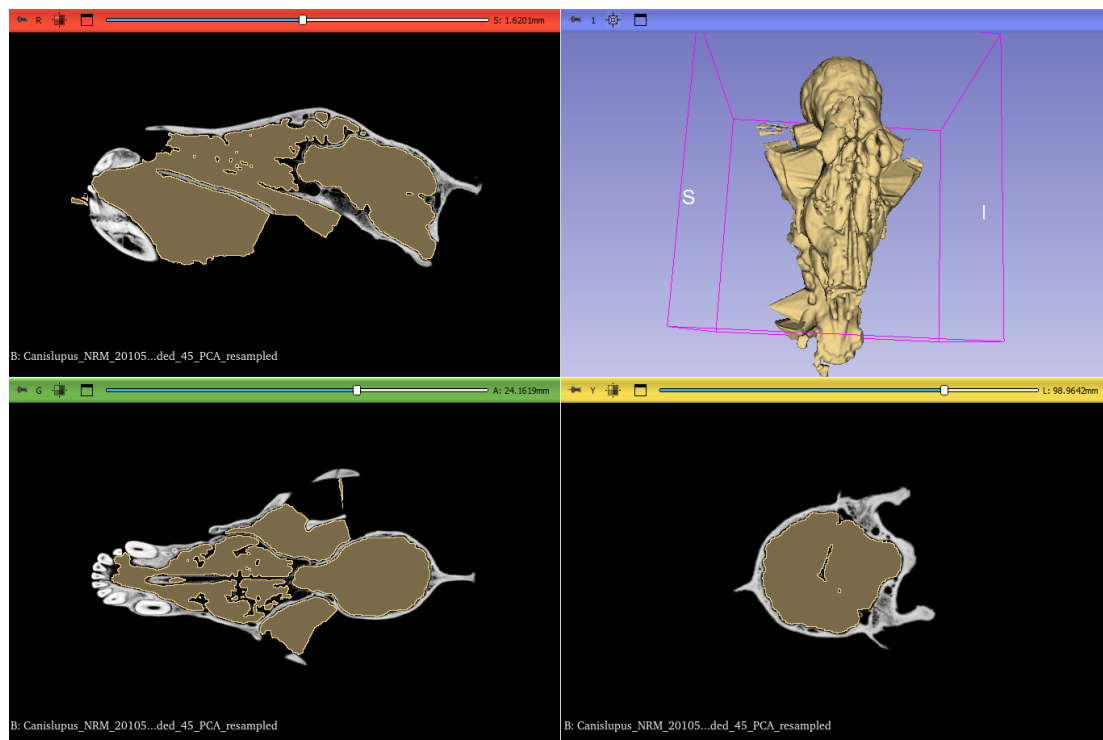


Figure 39: Smoothing kernel is 3 mm and maximum hole size is 10.

A.2 SlicerMorph result 2

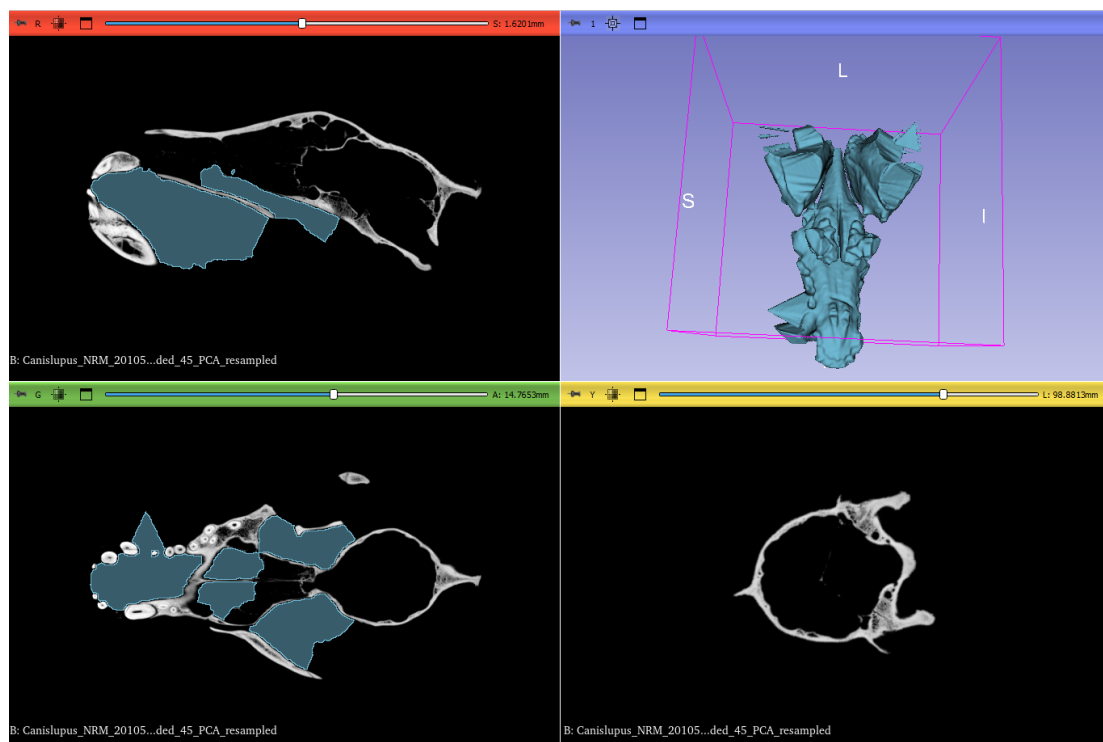


Figure 40: Smoothing kernel is 3 mm and maximum hole size is 2.

A.3 SlicerMorph result 3

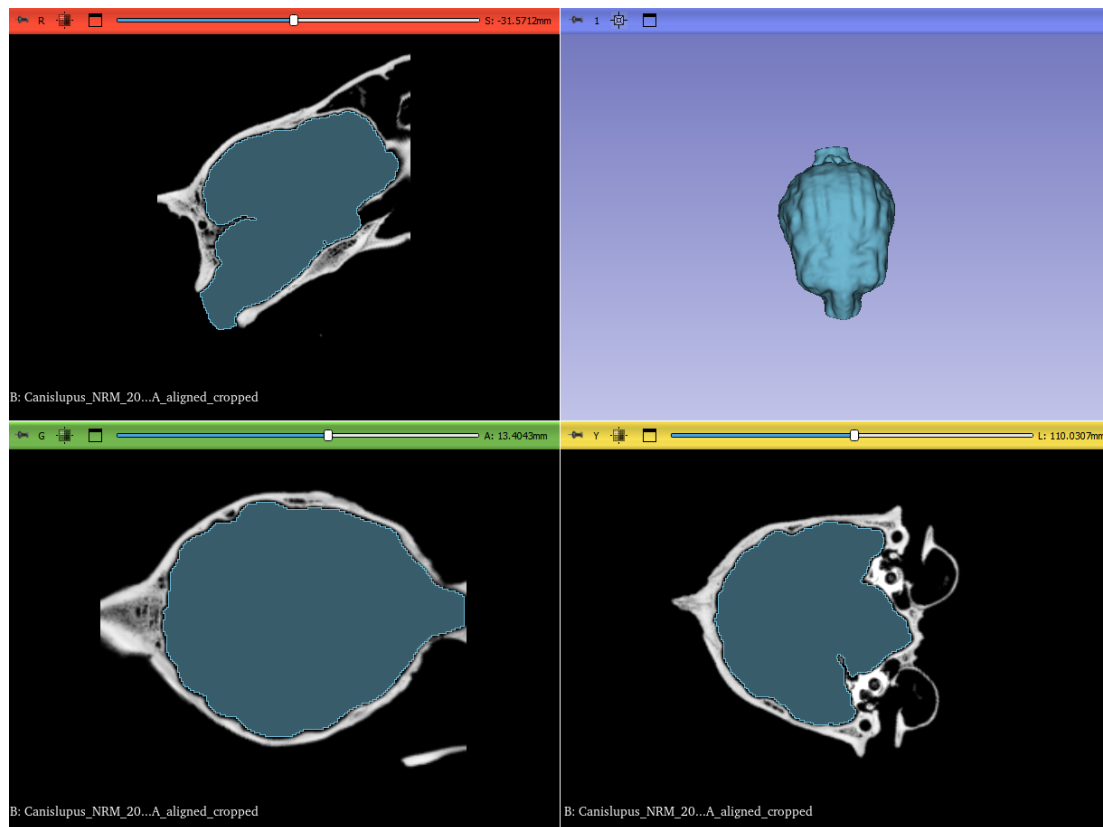


Figure 41: Made by doing one segment with 3 mm smoothing kernel and 10 mm maximum hole size, then a new segment with 3 mm smoothing kernel and 7 mm maximum hole size. It can clearly be seen that the olfactory bulb is missing

B Results

B.1 Settings used in time comparison table

Wolf_500

num_iters = [5, 3, 3]

step_sizes = [1, 0.5, 0.5]

$\Delta i = 15$ when icosphere is initialized, then 2 during first subdivision and 1 during the last 2 subdivision.

Wolf_1000

num_iters = [10, 5, 3, 2]

step_sizes = [1, 1, 0.5, 0.5]

$\Delta i = 10$ when icosphere is initialized, then 4 during first subdivision and 1 during the last 3 subdivision.

Wolf_1000_V2

num_iters = [10, 5, 3, 2, 1]

step_sizes = [1, 1, 0.5, 0.5, 0.5]

$\Delta i = 10$ when icosphere is initialized, then 4 during first subdivision and 1 during the last 4 subdivision.